

摘要

本文研究塔式太阳能光热电站定日镜场的光学性能评价与优化设计。我们统一建立太阳位置、镜面跟踪、阴影遮挡、有限圆柱接收和热功率模型，并在同一评价口径下依次求解给定镜场评价、统一规格镜场设计和逐镜可变规格镜场设计。

针对问题一，我们根据太阳高度角与方位角确定入射方向，以太阳方向和接收器方向的角平分线确定逐镜法向；随后采用**镜面点-pillbox 太阳盘方向耦合的 Sobol 低差异光线追迹**，在同一光线上依次判断邻镜及塔身阴影、出射遮挡和接收器命中。两级分辨率下年平均功率的相对差为 0.0134%，月度单位面积功率的最大相对差为 0.0594%；太阳半张角上下扰动 5% 时，年平均功率相对变化均不超过 0.33%，表明主要结果对数值分辨率和太阳盘参数的小幅扰动较为稳定。给定 1745 面镜场的**年平均光学效率为 0.5723，年平均输出热功率为 34.9868 MW，单位面积功率为 0.5569 kW/m²**。

针对问题二，我们以年平均功率不低于 60 MW 为硬约束，以单位镜面面积年平均输出最大为目标，采用**三角密排参数化与粒子群算法联合搜索塔位、统一镜面规格、安装高度和布局参数**。两级分辨率下年平均功率相对变化为 0.0109%，月度单位面积功率的最大相对变化为 0.0824%，优化结果具有较好的数值稳定性。最终布置 2983 面 6.7083×6.7083 m 定日镜，塔位为 (0, -33.0924) m，安装高度为 3.8926 m，总镜面面积为 134238.7113 m²，最小镜心间距为 11.7083 m；**年平均功率为 60.0257 MW，单位面积功率为 0.4472 kW/m²**。

针对问题三，我们将镜数、镜位以及逐镜宽度、高度和安装高度同时放开，构造 [2, 8] m 连续编码的两区密排遗传搜索，并以差分进化进行逐镜连续细化。为避免中精度评价在 60 MW 边界上的乐观偏差，再通过连续插值保留正式容量裕量。两级分辨率下年平均功率相对变化为 0.0450%，月度单位面积功率的最大相对变化为 0.1413%，连续规格方案在正式复核精度下保持稳定。最终方案含 2939 面镜，塔位为 (0, -37.4265) m，总面积为 132952.1202 m²，镜宽、镜高和安装高度分别分布于 4.7554–6.8000 m、4.7482–6.8000 m 和 3.4091–4.8225 m；**年平均功率为 60.1119 MW，单位面积功率为 0.4521 kW/m²**，相对问题二提高 1.1127%。

结果表明，统一的三维光线评价器能够支撑不同设计自由度下的可比优化；在记录的搜索预算和参数化布局族内，逐镜连续规格进一步减少了低贡献镜面面积。本文所得方案均通过几何约束、嵌套分辨率和多种子复核，但不作全局最优或真实工程净发电量的外推。

关键词： 定日镜场 Sobol 低差异光线追迹 粒子群算法 遗传算法 差分进化

一、问题重述

题目计划在东经 98.5° 、北纬 39.4° 、海拔 3000 m 处建设半径为 350 m 的圆形定日镜场。镜场坐标系以建设区域中心为原点， x 轴向东、 y 轴向北、 z 轴竖直向上。规划吸收塔高度为 80 m，塔顶集热器为高 8 m、直径 7 m 的圆柱形外表受光式集热器，吸收塔周围 100 m 内不得布置定日镜。

定日镜为宽度不小于高度的矩形，边长均须位于 2–8 m，安装高度位于 2–6 m，并保证镜面转动时不触地。相邻定日镜底座中心距离应比镜面宽度至少多 5 m。题目规定年均指标使用每月 21 日 9:00、10:30、12:00、13:30 和 15:00 五个时点计算。

问题一：吸收塔位于区域中心，全部定日镜尺寸为 $6\text{ m} \times 6\text{ m}$ 、安装高度为 4 m，镜心位置由附件给定。计算月平均和年平均光学效率、输出热功率及单位镜面面积输出热功率。

问题二：在全部定日镜尺寸及安装高度相同的条件下，设计吸收塔位置、镜面尺寸、安装高度、镜数和镜位，使年平均输出热功率达到 60 MW，并使单位镜面面积年平均输出热功率尽量大。

问题三：允许各定日镜尺寸和安装高度不同，在相同额定功率条件下重新设计镜场，并继续提高单位镜面面积年平均输出热功率。

二、总体分析

三个问题共享同一条计算主线。我们首先在问题一中建立可复核的光学评价内核：由日期和时点确定太阳方向与法向直接辐射辐照度，再由太阳入射方向和镜心指向集热器中心的方向确定定日镜姿态，分别计算余弦、大气透射、阴影遮挡、截断和反射效率，最后汇总为镜场热功率与月、年指标。

在问题二和问题三中，我们不改变评价口径，而把问题一的计算内核作为候选镜场的统一评价器。问题二采用统一镜面尺寸与安装高度；问题三进一步放开镜数、镜位以及逐镜尺寸和高度。两问均需同时处理场界、塔周禁布区、旋转离地、镜间距和 60 MW 额定功率约束。

因此，我们按“统一评价内核 → 约束可行性 → 候选设计比较 → 高精度复算”的顺序组织全文。搜索阶段使用低成本近似筛选候选，最终表格、Excel 提交文件和论文数值则统一来自正式高精度结果。

数值方法上，我们用耦合低差异光线追迹求解问题一的有限镜面与有限接收器积分；问题二采用参数化密排与粒子群算法，问题三采用连续遗传搜索与逐镜差分进化。

三个问题最终都沿用同一套太阳角、反射、阴影遮挡、有限圆柱截断和功率聚合公式，从而保证结果可以直接比较。

三、模型假设

三个问题使用同一光学评价器，问题二和问题三只改变设计变量与约束，并不重新定义太阳辐射、反射和能量聚合方式。因此，下列假设同时适用于三问；坐标范围、给定镜场规模和集热器尺寸等题面事实不列作假设。

1. **全年晴空假设。**假设全年规定评价时段均为晴天，不考虑云层对太阳直射辐射的随机遮蔽，法向直接辐射强度统一由题目给定的 DNI 公式计算。该假设使各月规定时刻能够在相同气象口径下比较，但所得年平均功率不包含阴雨天气和云量变化造成的实际可利用率损失。

2. **理想平面镜假设。**定日镜没有题面未给出的面形、斜率和跟踪误差，镜面法向使太阳中心光线指向集热器中心。该假设使反射几何可唯一确定，但真实电站的截断效率可能更低。

3. **均匀太阳盘假设。**太阳锥采用半角 $4.65 \times 10^{-3} \text{ rad}$ 、单位立体角能量均匀的 pillbox 分布。题面没有提供周日比，故不额外引入需要经验参数的周日区模型；问题一再半角 $\pm 5\%$ 检验该简化对功率的影响。

4. **联合光线计损假设。**阴影和遮挡均按每条光线的布尔可见性判断，多镜相交不重复计损；截断效率以未受阴影遮挡的反射光为条件分母。这是使题面效率乘积符合能量守恒的必要口径。

5. **离散年平均假设。**每月五个规定时点等权形成月平均，十二个月等权形成题面离散年平均。三问均采用这一口径，使评价与优化目标一致；它不等同于逐日全年积分，问题一另与按月天数加权结果比较。

6. **塔身与集热器几何。**三问均把塔身表示为半径 3.5 m、 $0 \leq z < 72 \text{ m}$ 的实体圆柱并计入入射阴影；集热器侧面取 $72 \leq z \leq 80 \text{ m}$ ，只在反射后的命中判定中使用。到达集热器侧面的反射能量按题面公式计入输出热功率，不额外加入未给出的吸收率或热损失。

四、符号说明

表 1 三个问题共用的太阳与光学模型符号

符号	含义	单位
α_s	太阳高度角	rad
γ_s	太阳方位角	rad
φ	当地纬度，北纬为正	rad
ω	太阳时角	rad
δ	太阳赤纬角	rad
D	以春分为第 0 天起算的天数	d
DNI	法向直接辐射辐照度	kW/m ²
N	定日镜总数	面
A_i	第 i 面定日镜采光面积	m ²
η_i	第 i 面定日镜光学效率	—
E_{field}	定日镜场输出热功率	kW
η_{sb}	阴影遮挡效率	—
η_{cos}	余弦效率	—
η_{at}	大气透射率	—
η_{trunc}	集热器截断效率	—
η_{ref}	镜面反射率	—
d_{HR}	镜面中心到集热器中心的距离	m
$\mathbf{s}, \mathbf{s}'$	太阳中心方向及太阳盘内方向单位向量	—
$\mathbf{t}_i, \mathbf{n}_i$	第 i 面镜指向接收器的方向及镜面单位法向	—
β_i, τ_i	第 i 面镜法向方位角及镜面相对水平面的倾角	rad
$\mathbf{e}_{w,i}, \mathbf{e}_{h,i}$	第 i 面镜局部宽向、高向单位向量	—
$\theta_{\odot}$	均匀太阳盘半角	rad
J_i, T_i	入射光相对其他镜面、塔身的可见指示量	—
I_i, O_i, H_i	联合入射可见、出射可见及集热器命中指示量	—
$\mathcal{C}_T$	半径 3.5 m、高 72 m 的塔身实体圆柱	m 坐标集合
$\mathcal{M}_i, \Omega_i$	有限矩形镜面及镜面点-太阳方向联合样本空间	—
P_A	单位镜面面积年平均输出热功率	kW/m ²

表 2 问题二与问题三的主要优化符号

符号	含义	单位
k	题目规定的太阳状态序号, $k = 1, \dots, 60$	—
DNI_k, D_k	第 k 个太阳状态的法向直接辐射辐照度	kW/m^2
(x_T, y_T)	吸收塔在建设场地内的平面坐标	m
(x_i, y_i, z_i)	问题三第 i 面定日镜的镜心坐标	m
w, h, z_H	问题二全场统一的镜宽、镜高和安装高度	m
w_i, h_i, a_i	问题三第 i 面镜的宽度、高度及面积, $a_i = w_i h_i$	m, m, m^2
A_{tot}	镜场总采光面积	m^2
$P_k, \overline{P}_{\text{field}}, \overline{P}$	状态输出功率及问题二、三的年平均输出热功率	MW
$\mathcal{D}, \mathcal{D}_3$	问题二、三的完整设计变量集合	—
d_{ij}	第 i, j 面定日镜的镜心距离	m
R_b	问题三内、外径向区域的分界半径	m
ϕ_q	区域 q 的三角点阵纵向平移相位, 占一个点阵行距的比例	周期
$d_q, \Delta y_q$	区域 q 的三角点阵最近邻间距与相邻行距	m
ρ_q	区域 q 的基础镜面高宽比, $\rho_q = h_q/w_q$	—
c	镜面下缘相对地面的净空参数	m
α	问题三容量修复时两组逐镜规格之间的插值系数	—

五、问题一：给定镜场的光学性能评价

5.1 问题分析与建模主线

问题一并非只需把题面给出的五项效率代入乘积公式。余弦效率取决于每面镜的三维姿态，阴影与遮挡取决于光线在反射前后的有向路径，截断效率又取决于太阳并非单一平行方向以及接收器具有有限尺寸。因此，必须先定义从太阳辐射到圆柱接收器接收能量的连续物理映射，再选择数值方法求解其中的几何积分。

本问固定吸收塔平面坐标为 $(0, 0)$ ，按修订几何口径取集热器中心为 $(0, 0, 76)$ m。附件给出 1745 面定日镜的镜心平面坐标；每面镜为 $6 \text{ m} \times 6 \text{ m}$ ，镜心高度均为 4 m ，总采光面积为 62820 m^2 。题目规定每月 21 日的 9:00、10:30、12:00、13:30 和 15:00 作为代表时点，故评价集合共有 $12 \times 5 = 60$ 个状态。

本问按“太阳输入 → 镜面几何 → 联合样本 → 有向光路事件 → 条件效率 → 逐状态功率与聚合”的顺序建模。首先由日期和时刻计算太阳方向与 DNI，为姿态和能量计算提供统一输入；再由太阳方向与接收器方向确定逐镜法向，并以有限矩形表示真实镜

面边界，使余弦损失和光线相交均有明确的三维几何对象。

在此基础上，将镜面采样点与太阳盘方向组成联合样本，使阴影、遮挡和截断判定始终作用于同一份入射能量。光线依次经历入射可见性判断、镜面反射、出射可见性判断和有限接收器命中判断，再由联合事件比例得到阴影遮挡效率与条件截断效率。最后逐镜、逐时点计算输出功率，并依次聚合为月平均和年平均指标。该顺序保证各损失不会重复扣除，同时保留 DNI 与光学效率的同步变化；数值采样只用于求解其中的几何积分，不改变各效率的物理定义。

5.2 坐标系、基本假设与适用范围

建立右手直角坐标系： x 轴向东， y 轴向北， z 轴竖直向上。太阳方位角从正北顺时针量取。按本文采用的几何口径，塔身是半径 3.5 m、 $0 \leq z < 72$ m 的实体圆柱；集热器侧面为半径 3.5 m、 $72 \leq z \leq 80$ m 的有限圆柱侧面，中心为 (0, 0, 76)。阴影表示入射光到达目标镜之前被其他镜面或塔身截断，遮挡表示反射光到达接收器区域之前被其他镜面截断；同一条光线即使与多个物体相交，也只记一次损失。

在题面没有给出额外参数的部分，我们采用以下简化：定日镜为理想平面镜，不加入面形误差、斜率误差和跟踪误差；太阳锥采用半角 $\theta_{\odot} = 4.65 \times 10^{-3}$ rad 的均匀 pillbox 太阳盘；五个时点等权形成月平均，十二个月等权形成题面离散年平均；到达圆柱侧面的反射能量按题面公式全部计入输出热功率，不再额外乘入接收器吸收率。

因此，后文“年平均”专指 60 个规定状态的等权离散平均，不是逐日逐小时气象积分；所得光学性能也只对应理想平面镜和均匀 pillbox 太阳盘，不直接等同于包含污染、跟踪误差和热损失的实际电站年发电量。

5.3 太阳位置与 DNI

确定坐标和适用范围后，首先把题目规定的日期与时刻转换为太阳方向和辐射强度。二者共同构成后续逐镜姿态与能量计算的时变输入。

令状态集合为

$$\mathcal{T} = \{(m, h) : m = 1, \dots, 12; h \in \{9, 10.5, 12, 13.5, 15\}\}. \quad (1)$$

对 $t = (m, h) \in \mathcal{T}$ ，记每月 21 日相对春分的天数为 D_m ，当地纬度为 $\varphi = 39.4^\circ$ 。根据题面附录，赤纬角、时角与高度角依次满足

$$\sin \delta_m = \sin \left(\frac{2\pi D_m}{365} \right) \sin 23.45^\circ, \quad \omega_h = \frac{\pi}{12}(h - 12), \quad (2)$$

$$\sin \alpha_s(t) = \cos \delta_m \cos \varphi \cos \omega_h + \sin \delta_m \sin \varphi. \quad (3)$$

题面只给出方位角余弦，反余弦本身不能区分上午和下午。先计算

$$C_\gamma(t) = \frac{\sin \delta_m - \sin \alpha_s(t) \sin \varphi}{\cos \alpha_s(t) \cos \varphi}, \quad (4)$$

再利用时角补足象限：

$$\gamma_s(t) = \begin{cases} \arccos C_\gamma(t), & \omega_h \leq 0, \\ 2\pi - \arccos C_\gamma(t), & \omega_h > 0. \end{cases} \quad (5)$$

这样定义后，上午太阳位于东侧，下午位于西侧，正午位于南侧。镜场指向太阳中心的单位向量为

$$\mathbf{s}(t) = \begin{bmatrix} \cos \alpha_s(t) \sin \gamma_s(t) \\ \cos \alpha_s(t) \cos \gamma_s(t) \\ \sin \alpha_s(t) \end{bmatrix}. \quad (6)$$

太阳高度角和方位角分别控制太阳方向的铅垂分量与水平投影。图1把三维方向拆成水平面和铅垂面两个视图，以明确方位角从正北顺时针的约定和高度角相对于地平面的定义。

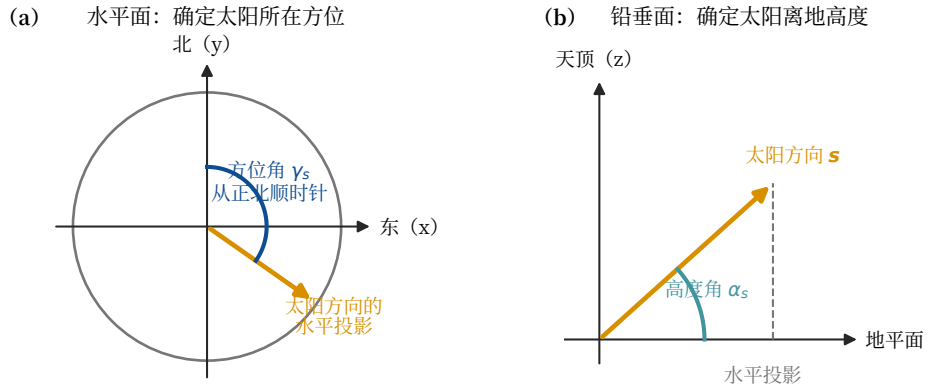


图1 太阳高度角、方位角及太阳中心向量的坐标约定

图1(a) 解释了为何方位角必须结合时角确定象限；图1(b) 说明 $\sin \alpha_s$ 就是太阳单位向量的竖直分量。太阳向量既用于后续镜面姿态和阴影光路，也出现在 DNI 公式中，因此太阳角是整个模型的第一项承重输入。

题目给定海拔 $H = 3 \text{ km}$ ，代入附录参数后，法向直接辐射辐照度为

$$\text{DNI}(t) = 1.366 \left[0.34981 + 0.5783875 \exp \left(-\frac{0.275745}{\sin \alpha_s(t)} \right) \right] \text{ kW/m}^2. \quad (7)$$

DNI 给出垂直于太阳中心光线平面的能量通量， $\mathbf{s}$ 给出光线方向。余弦效率将在镜面上完成投影，因此不能把 DNI 再乘一次水平面投影因子。

在把 60 个状态输入镜场模型之前，需要确认它们都是有效白昼状态，并检查季节和时刻变化是否符合球面几何。图2分别给出 60 个状态的太阳高度角和 DNI；每个单元格都对应后续全场计算中的一个时点。

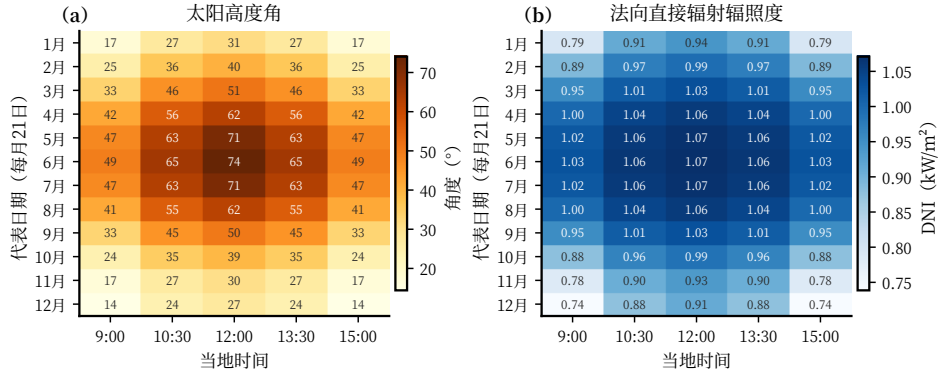


图2 题目规定 60 个状态的太阳高度角与法向直接辐射辐照度

图2表明 60 个状态的太阳高度角和 DNI 均为正；同月内正午最高，同一时刻由冬季向夏季整体升高。上午和下午关于正午具有相同的高度角与 DNI，但方位角处于不同象限，所以镜场空间损失仍可能不同，不能只凭 DNI 对称性推断镜场效率完全对称。

5.4 反射定律与镜面姿态

太阳方向确定后，需要进一步求出每面镜在各时点的目标姿态，使太阳中心光线经过反射后指向集热器中心。

第 i 面镜的镜心为 $\mathbf{c}_i = (x_i, y_i, 4)^\top$ ，集热器中心为 $\mathbf{c}_R = (0, 0, 76)^\top$ 。镜心指向集热器中心的单位向量为

$$\mathbf{t}_i = \frac{\mathbf{c}_R - \mathbf{c}_i}{\|\mathbf{c}_R - \mathbf{c}_i\|}. \quad (8)$$

太阳中心光线的传播方向是 $-\mathbf{s}$ 。理想镜面反射算子为

$$\mathcal{R}(\mathbf{d}, \mathbf{n}) = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}. \quad (9)$$

控制规则要求 $\mathcal{R}(-\mathbf{s}, \mathbf{n}_i) = \mathbf{t}_i$ ，由入射方向与目标方向的角平分关系得到唯一朝阳法向

$$\mathbf{n}_i(t) = \frac{\mathbf{s}(t) + \mathbf{t}_i}{\|\mathbf{s}(t) + \mathbf{t}_i\|}. \quad (10)$$

该式给出的不是固定安装角，而是随太阳位置更新的目标姿态。对固定的第 i 面镜， $\mathbf{t}_i$ 在一天内保持不变；每到一个计算时点，先更新 $\mathbf{s}(t)$ ，再由上式更新 $\mathbf{n}_i(t)$ 。因此，同一面镜上午、正午和下午需要转到不同姿态；同一时点的不同镜面也因 $\mathbf{t}_i$ 不同而具有不同姿态。

为直观表示控制角，定义法向方位角 β_i 从正北顺时针量取，镜面倾角 τ_i 为镜面平面相对水平面的夹角，则

$$\beta_i(t) = \text{mod}[\text{atan2}(n_{x,i}(t), n_{y,i}(t)), 2\pi], \quad \tau_i(t) = \arccos n_{z,i}(t). \quad (11)$$

当 $\tau_i = 0$ 时镜面水平。这两个角确定题面模型中的目标镜面平面；实际电机轴角还依赖支架零位和传动形式，题面没有提供，故不另行虚构。

以附件第1面镜 $\mathbf{c}_1 = (107.25, 11.664, 4)^\top \text{ m}$ 为例，图3给出3月21日五个时点的目标姿态。该镜位于塔的东偏北侧，因此即使上午、下午太阳高度角关于正午对称，其镜面姿态也不必关于正午对称。

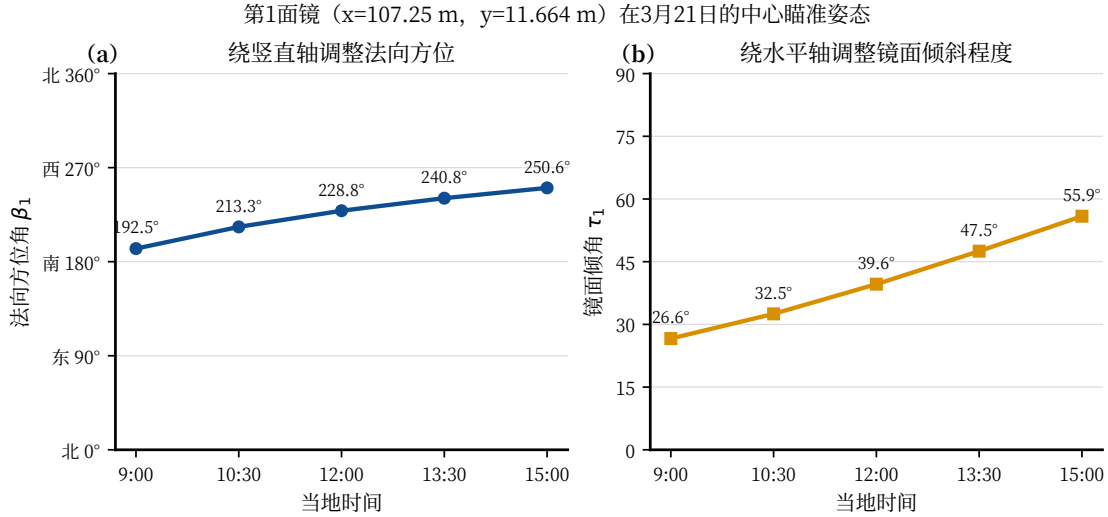


图3 附件第1面定日镜在3月21日的时变目标姿态

这里“使光线尽量反射至集热器”严格对应题面中心瞄准规则：太阳盘中心光线经镜心反射后指向集热器中心。它不是额外的瞄准点优化，也不意味着太阳锥中的所有边缘光线都必然命中有限接收器；后者仍由截断效率计算。

因此余弦效率是反射几何的解析结果：

$$\eta_{\cos,i}(t) = \mathbf{n}_i(t) \cdot \mathbf{s}(t) = \sqrt{\frac{1 + \mathbf{s}(t) \cdot \mathbf{t}_i}{2}}. \quad (12)$$

法向只确定镜面所在平面，尚未给出有限矩形边界。令 $\mathbf{k} = (0, 0, 1)^\top$ ，根据“上下边平行地面”的题意构造

$$\mathbf{e}_{w,i} = \frac{\mathbf{k} \times \mathbf{n}_i}{\|\mathbf{k} \times \mathbf{n}_i\|}, \quad \mathbf{e}_{h,i} = \mathbf{n}_i \times \mathbf{e}_{w,i}. \quad (13)$$

于是有限矩形镜面为

$$\mathcal{M}_i(t) = \{\mathbf{c}_i + u\mathbf{e}_{w,i}(t) + v\mathbf{e}_{h,i}(t) : u, v \in [-3, 3]\}. \quad (14)$$

这一步需要同时说明反射方向和镜面边界，因为后续光线相交既依赖法向，又必须检查交点是否真正落在 $6 \text{ m} \times 6 \text{ m}$ 矩形内部。图4左图表示法向的角平分关系，右图表示局部宽向、高向和法向如何共同参数化有限镜面。

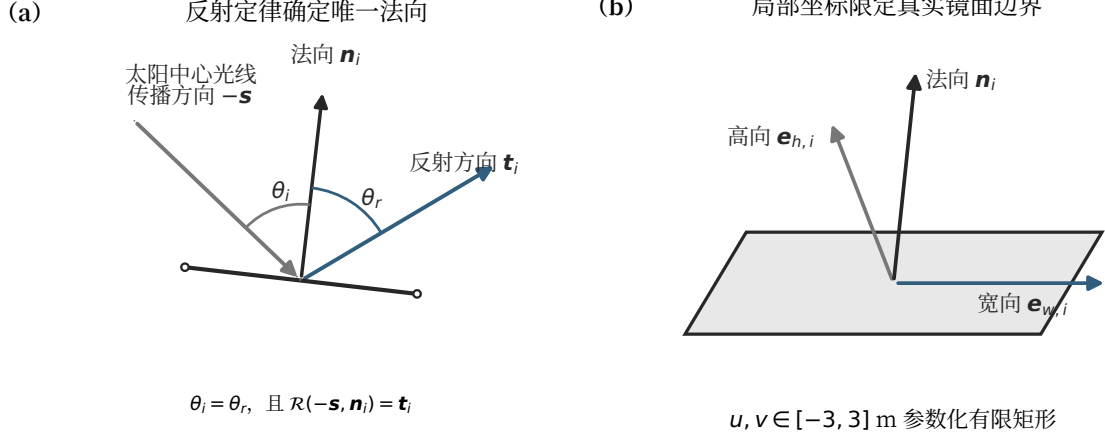


图4 镜面反射方向、唯一法向与有限矩形局部坐标

图4(a) 与法向公式对应: n_i 位于指向太阳的方向 s 和指向接收器的方向 t_i 之间; 图4(b) 与有限镜面参数式对应: 只有同时满足 $u, v \in [-3, 3]$ 的交点才位于真实镜面上。至此, 太阳位置被转换成每个时点、每面镜的唯一三维有限几何对象。

5.5 太阳盘方向测度

镜面法向只保证太阳中心光线命中接收器中心, 但真实太阳具有有限视角, 因而截断效率还取决于太阳盘内不同方向。pillbox 模型把太阳表示为角半径内亮度均匀、边界外亮度为零的圆盘, 是聚光太阳能光线追迹中常用的基础测试太阳形状 [3]。由于题面没有给出周日比或现场太阳形状, 我们采用半角 $\theta_{\odot} = 4.65 \times 10^{-3}$ rad 的均匀 pillbox 太阳盘。在 s 的正交平面内构造单位基 a, b , 太阳盘内方向表示为

$$s'(t, \theta, \psi) = s(t) \cos \theta + [a(t) \cos \psi + b(t) \sin \psi] \sin \theta, \quad (15)$$

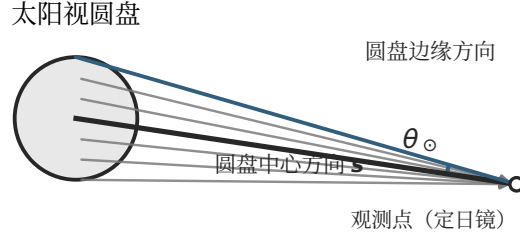
其中 $0 \leq \theta \leq \theta_{\odot}$, $0 \leq \psi < 2\pi$ 。均匀 pillbox 是单位立体角内能量相同, 因此归一化方向测度为

$$d\mu_s = \frac{\sin \theta d\theta d\psi}{2\pi(1 - \cos \theta_{\odot})}. \quad (16)$$

数值采样必须使 $\cos \theta$ 均匀, 而不能使 θ 均匀; 否则会过度采样太阳盘中心, 系统改变接收器边界附近的命中比例。

图5从定日镜观测点连接太阳视圆盘的中心和边缘, 直观给出半张角 $\theta_{\odot}$ 的几何含义。图中光线由太阳盘传播至定日镜, 而太阳中心方向 s 由定日镜指向圆盘中心; 为使夹角可辨, 图示角度相对于真实值作了放大。

太阳视圆盘对应的入射方向集合



视半径: $\theta_{\odot} = 4.65 \text{ mrad} \approx 0.266^\circ$
视直径: $2\theta_{\odot} \approx 0.533^\circ$

角度为示意放大, 非实际比例

图 5 太阳视圆盘、中心方向与半张角 $\theta_{\odot}$

镜面法向仍由太阳中心方向确定, 但太阳盘内每个方向都要逐条反射:

$$\mathbf{r}'_i = -\mathbf{s}' + 2(\mathbf{s}' \cdot \mathbf{n}_i)\mathbf{n}_i. \quad (17)$$

只有 $\mathbf{s}' = \mathbf{s}$ 时, 反射方向才严格等于 $\mathbf{t}_i$. 因此, 中心光线命中接收器中心并不能替代对整个太阳锥的有限接收器命中计算。

5.6 阴影、遮挡与截断的统一光线判定

有了有限镜面和太阳盘方向后, 便可以在同一光线样本上依次定义入射阴影、反射后遮挡与接收器命中。统一样本空间使各项损失具有一致分母, 避免对同一份能量重复扣减。

对第 i 面镜和时点 t , 定义联合样本空间

$$\Omega_i(t) = \mathcal{M}_i(t) \times \mathcal{S}(t), \quad d\mu_i = \frac{dA}{A_i} d\mu_s, \quad A_i = 36 \text{ m}^2. \quad (18)$$

一个样本 $\xi = (\mathbf{p}, \mathbf{s}')$ 同时指定镜面位置和太阳盘方向, 代表一小份确定光路的能量。阴影、遮挡和截断均在同一个联合样本空间上判断, 才能保证损失分母一致。

入射阶段从镜面点朝太阳方向回溯:

$$\mathbf{q}_{\text{in}}(\lambda) = \mathbf{p} + \lambda \mathbf{s}', \quad \lambda > 0. \quad (19)$$

先定义塔身实体区域

$$\mathcal{C}_T = \{(x, y, z) : x^2 + y^2 \leq 3.5^2, 0 \leq z < 72\}. \quad (20)$$

若入射射线不与任何其他有限镜面相交，令 $J_i(\xi) = 1$ ，否则为 0；若它不与 $\mathcal{C}_T$ 相交，令 $T_i(\xi) = 1$ ，否则为 0。于是联合入射可见指示量为

$$I_i(\xi) = J_i(\xi)T_i(\xi). \quad (21)$$

这样，镜间阴影和塔身阴影在同一条光线上作布尔合并，不会重复扣损。 $72 \leq z \leq 80$ m 的集热器不作为入射阴影体；该区间只在反射后的接收命中判定中使用。出射阶段沿反射方向追踪

$$\mathbf{q}_{\text{out}}(\lambda) = \mathbf{p} + \lambda \mathbf{r}'_i, \quad \lambda > 0. \quad (22)$$

若反射光在到达接收器区域之前先与其他镜面相交，则发生遮挡。定义出射可见指示量 $O_i(\xi)$ ：未被遮挡时取 1，否则取 0。

阴影和遮挡发生在反射前后不同的有向光段。图6分别给出两类光路，以避免把它们混作无方向的平面重叠面积。

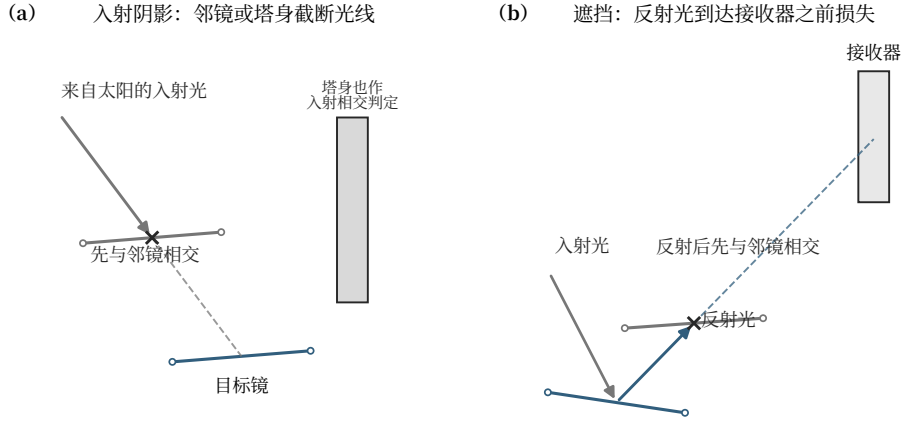


图 6 入射阴影与出射遮挡的有向光路区别

图6(a) 中光线尚未到达目标镜就与邻镜或塔身相交；图6(b) 中太阳光已到达目标镜，但反射后在到达接收器前与邻镜相交。程序因此分别沿 $+\mathbf{s}'$ 和 $+\mathbf{r}'$ 检查正向射线参数，不能用同一个反向投影公式代替两类事件。

为单独量化塔身带来的入射损失，对第 i 面镜定义 60 状态累计塔影暴露量

$$N_{T,i}^{\text{eq}} = \sum_{t \in \mathcal{T}} \Pr_{\mathbf{p}, \mathbf{s}', \text{seed}} \{ \mathbf{q}_{\text{in}}(\lambda) \cap \mathcal{C}_T \neq \emptyset \}. \quad (23)$$

每个状态中的概率在 64 个镜面点、32 个太阳盘方向和 4 个固定种子上求平均。 $N_{T,i}^{\text{eq}} = 1$ 表示累计暴露量等价于一个规定状态下整面镜完全被塔身遮挡；该指标同时反映受影响状态数和每个状态的遮挡面积比例，但不混入镜间阴影。

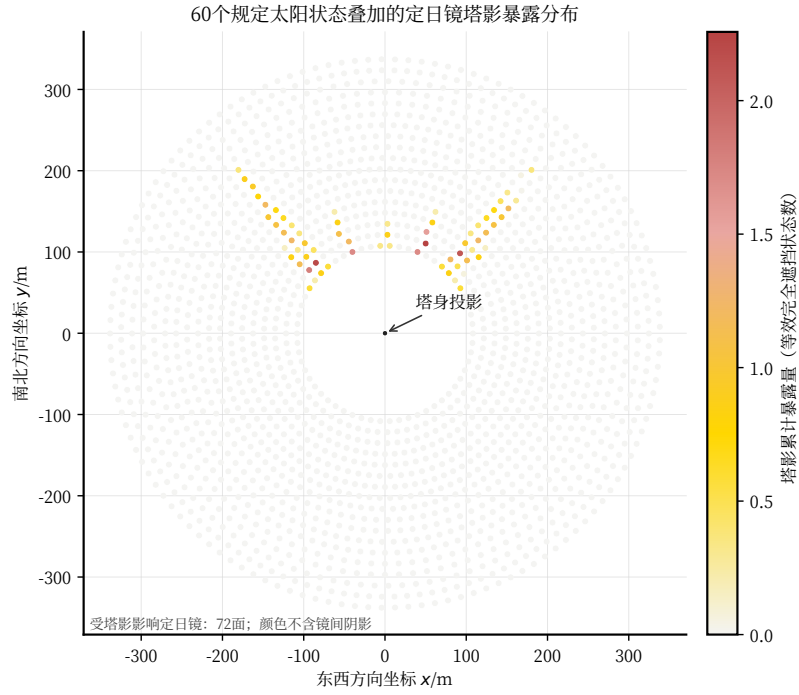


图7 60个规定太阳状态叠加的定日镜塔身阴影暴露分布

图7中颜色越深，表示累计塔影暴露越强。共有 72 面定日镜出现非零塔影，单镜最大累计暴露为 2.2574 个等效状态。受影响镜面主要位于塔的北侧，并向西北和东北展开。这是因为规定时点的太阳位于南侧天空，塔影总体投向北方，而上午、下午太阳方位变化使阴影方向发生横向偏转。这里的“全年叠加”特指题目规定 60 个状态的等权累计，不等同于真实全年逐分钟遮挡时长。

接收器有效受光面为

$$\mathcal{C} = \{(x, y, z) : x^2 + y^2 = R^2, 72 \leq z \leq 80\}, \quad R = 3.5 \text{ m}. \quad (24)$$

将出射射线代入圆柱方程，得到 $a\lambda^2 + b\lambda + c = 0$ ，其中

$$a = r_x'^2 + r_y'^2, \quad b = 2(p_x r_x' + p_y r_y'), \quad c = p_x^2 + p_y^2 - R^2. \quad (25)$$

取最小正根 λ_R ，再检查 $p_z + \lambda_R r_z' \in [72, 80]$ 。满足时定义命中指示量 $H_i(\xi) = 1$ ，否则为 0。题面指定外表受光式圆柱，故顶面和底面不计入接收面。

截断损失同时来自太阳盘方向展开和接收器有限直径、高度。图8表示从有限镜面不同位置出发的太阳盘方向光线经反射后，只有部分命中有限圆柱侧面。

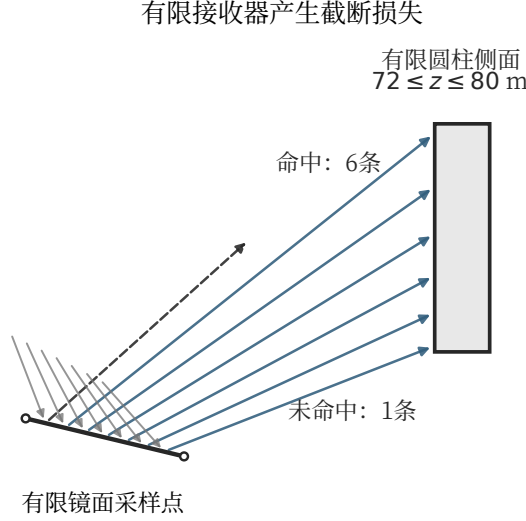


图 8 有限圆柱集热器的光线命中与截断机制

图8对应命中指示量 H_i 。中心光线命中只验证瞄准方向正确，不能说明 $\eta_{\text{trunc}} = 1$ ；太阳盘边缘方向和镜面不同位置仍可能形成未命中的反射光。

令光路未受入射阴影和出射遮挡的联合事件为 $V_i(\xi) = I_i(\xi)O_i(\xi)$ 。阴影遮挡效率定义为

$$\eta_{\text{sb},i}(t) = \int_{\Omega_i(t)} V_i(\xi) d\mu_i(\xi). \quad (26)$$

题面把截断效率的分母定义为扣除阴影遮挡损失后的反射能量，因此它必须是条件比例：

$$\eta_{\text{trunc},i}(t) = \frac{\int_{\Omega_i(t)} V_i(\xi) H_i(\xi) d\mu_i(\xi)}{\int_{\Omega_i(t)} V_i(\xi) d\mu_i(\xi)}. \quad (27)$$

于是严格满足

$$\eta_{\text{sb},i}(t) \eta_{\text{trunc},i}(t) = \int_{\Omega_i(t)} V_i(\xi) H_i(\xi) d\mu_i(\xi). \quad (28)$$

右端正是“既未被镜场阻挡，又真正命中接收器”的联合能量比例。该恒等式保证一条光线无论与多少邻镜相交都只记录一次损失，并保证截断效率不使用全部镜面采样作为错误分母。

5.7 单镜效率与镜场功率

完成三类光路事件的统一判定后，下一步把几何可见性与余弦效率、大气透射率和镜面反射率合并，形成逐镜功率并向镜场尺度聚合。

第 i 面镜到接收器中心的距离为 $d_{\text{HR},i} = \|\mathbf{c}_R - \mathbf{c}_i\|$ 。附件中全部 $d_{\text{HR},i} < 1000 \text{ m}$ ，故题面大气透射率公式适用：

$$\eta_{\text{at},i} = 0.99321 - 0.0001176d_{\text{HR},i} + 1.97 \times 10^{-8}d_{\text{HR},i}^2. \quad (29)$$

镜面反射率取 $\eta_{\text{ref}} = 0.92$ 。单镜综合光学效率为

$$\eta_i(t) = \eta_{\text{sb},i}(t)\eta_{\text{cos},i}(t)\eta_{\text{at},i}\eta_{\text{trunc},i}(t)\eta_{\text{ref}}. \quad (30)$$

单镜和镜场在时点 t 的输出热功率分别为

$$E_i(t) = \text{DNI}(t)A_i\eta_i(t), \quad (31)$$

$$E_{\text{field}}(t) = \sum_{i=1}^N E_i(t) = \text{DNI}(t) \sum_{i=1}^N A_i\eta_i(t). \quad (32)$$

Q1 全部镜面等面积，故时点平均效率可按 1745 面镜等权：

$$\bar{\eta}_q(t) = \frac{1}{N} \sum_{i=1}^N \eta_{q,i}(t), \quad q \in \{\text{sb}, \text{cos}, \text{trunc}, \text{total}\}. \quad (33)$$

月平均为当月五个时点的算术平均：

$$\bar{\eta}_{q,m} = \frac{1}{5} \sum_{h \in \{9, 10.5, 12, 13.5, 15\}} \bar{\eta}_q(m, h), \quad (34)$$

题面离散年平均为

$$\bar{\eta}_{q,\text{year}} = \frac{1}{12} \sum_{m=1}^{12} \bar{\eta}_{q,m} = \frac{1}{60} \sum_{t \in \mathcal{T}} \bar{\eta}_q(t). \quad (35)$$

功率也按相同时间权重聚合，但必须先计算每个时点的镜场功率，再求平均：

$$\bar{E}_{\text{year}} = \frac{1}{60} \sum_{t \in \mathcal{T}} \text{DNI}(t) \sum_{i=1}^N A_i\eta_i(t). \quad (36)$$

单位镜面面积年平均输出热功率为

$$P_A = \frac{\bar{E}_{\text{year}}}{\sum_i A_i}. \quad (37)$$

不能用“平均 DNI $\times$ 平均效率”替代上式，因为 DNI 和光学效率随状态共同变化，两个平均量的乘积一般不等于乘积的平均。

5.8 连续模型的数值求解

上述积分包含有限矩形可见性和有限圆柱命中，难以获得闭式解。我们分别在镜面局部坐标 (u, v) 和太阳盘立体角上生成随机扰动 Sobol 低差异样本，再计算两组样本的笛卡尔积。对每个“镜面点-太阳方向”组合依次计算 I_i, O_i, H_i ，用布尔样本均值估计阴影遮挡效率和联合接收比例，最后通过条件关系得到截断效率。

以 $N_M \times N_S$ 表示光线追迹分辨率，其中 N_M 是每面镜上的采样点数， N_S 是 pillbox 太阳盘内的方向采样数；二者作笛卡尔积，因此每面镜、每个时点、每个种子共追踪

$N_M N_S$ 条联合光线。细分分辨率取 64×32 ，即 2048 条联合光线；固定种子为 2023、2024、2025 和 2026，结果取四个估计的算术平均。为估计离散误差，同时使用 32×16 嵌套粗分辨率，其 Sobol 样本均为对应细序列的前缀，使粗细差异主要反映样本数量变化而非样本位置变化。

几何相交只需检查可能影响目标镜的邻镜。程序以镜心平面距离 60 m 建立候选集，但该剪枝必须与全 1745 面镜的相交判定进行分层复核。TensorFlow GPU 只负责分块执行相同的射线-有限矩形和射线-圆柱求交，不改变物理样本空间或效率定义。

64×32 结果相对 32×16 变化很小，故后续以 64×32 联合样本和四个固定种子的平均值作为数值结果。

5.9 几何验证与数值稳健性

模型验证对应建模链中的具体高风险环节，而不是只检查程序能否运行。除太阳方向、反射方向和集热器命中检查外，还构造了三类塔影案例：入射射线穿过塔身、背离塔身、仅穿过集热器高度区间，预期结果依次为“遮挡、不遮挡、不遮挡”。TensorFlow 与 NumPy 在独立复核案例中的相交计数完全一致；60 m 邻镜候选剪枝相对全场判定的阴影遮挡效率最大绝对差为 0。

在全部 $1745 \times 60 = 104700$ 条逐镜状态记录中，各效率均位于 $[0, 1]$ ，未出现 NaN 或无穷值；联合事件分解的最大残差处于浮点舍入量级，综合效率乘积残差为 0，镜场功率与逐镜功率汇总残差为 7.11×10^{-15} MW。这些结果分别验证了太阳角、反射方向、有限边界、条件截断分母和单位聚合。

进一步检验离散分辨率、随机化低差异采样和关键简化假设是否会明显改变结果。图9统一使用相对功率变化，并给出相应容差线，避免用截断的绝对功率纵轴夸大小差异。

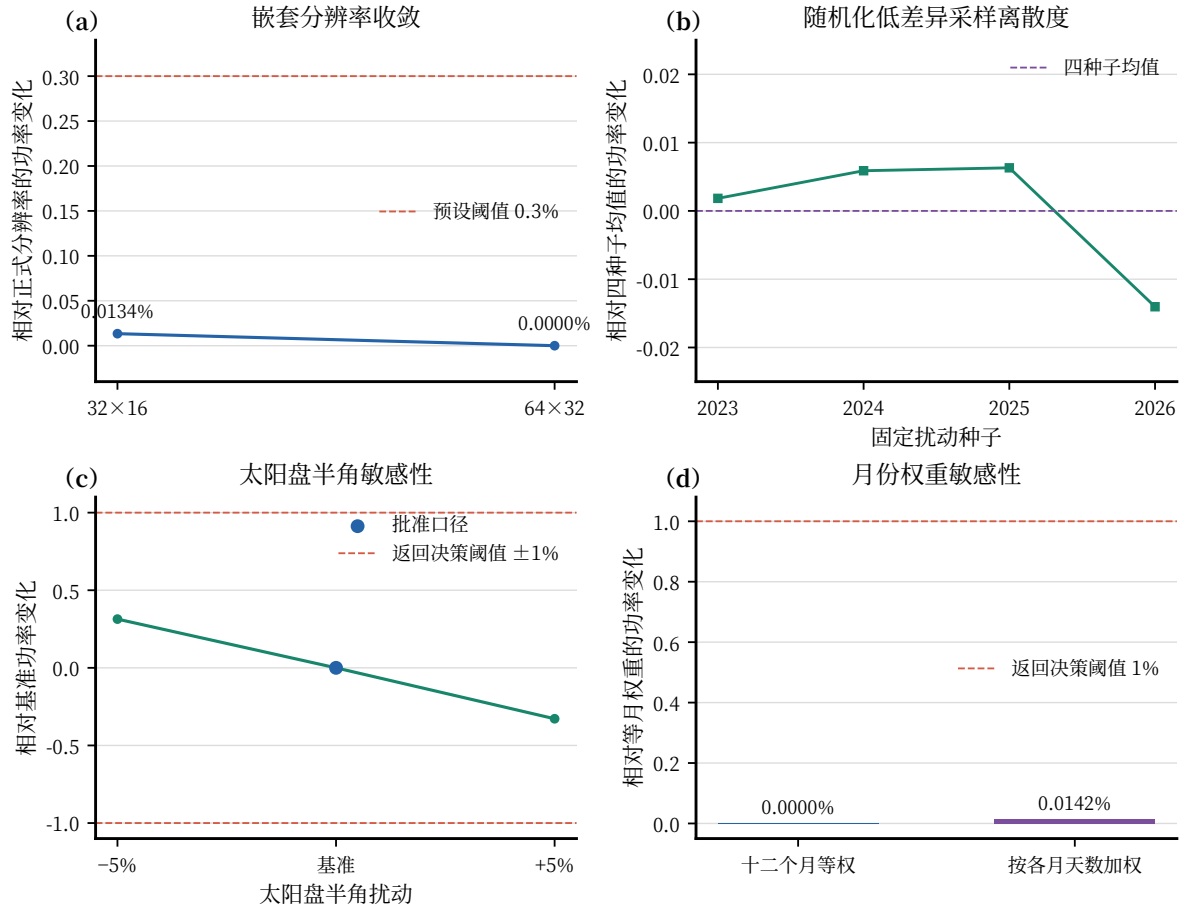


图9 光线追迹分辨率、固定种子及关键假设的稳健性检查

由图9(a)可见，从 32×16 提高到 64×32 后，年平均综合效率的绝对变化为 7.6896×10^{-5} ，年平均镜场功率相对变化为 0.0134%，12 个月单位面积功率的最大相对变化为 0.0594%；三项分别低于 0.002、0.3% 和 0.5% 阈值。图9(b) 中四种子年平均综合效率标准差为 4.6416×10^{-5} ，年平均功率极差仅 0.007117 MW。

图9(c) 显示，太阳盘半角改变 -5% 和 $+5\%$ 时，年平均功率分别改变 $+0.3140\%$ 和 -0.3281% ，均小于 1%；太阳锥变宽时反射光斑扩展，功率下降，方向与有限接收器几何一致。图9(d) 显示，按各月天数加权得到 34.9918 MW，相对十二个月等权的 34.9868 MW 仅变化 0.0142%。

综上，提高采样分辨率、改变低差异序列种子、调整太阳盘半角以及改变月份权重时，年平均功率均只发生较小变化，说明问题一结果在上述数值设置和假设扰动范围内较为稳定。该结论反映的是模型内部的数值稳健性，并不替代真实镜场的实测验证。

5.10 镜场空间分布与月度规律

全场平均会掩盖不同镜位与太阳时刻的差异。为了说明损失并非一个可对所有镜面套用的常数，图10先给出附件镜场布局；随后对每面镜的 60 个规定状态综合效率取等

权平均，形成逐镜年均综合光学效率分布；最后选择冬季上午、春季正午和夏季正午三个代表状态展示逐镜瞬时综合效率。年均图与三个时点图使用相同色标，因此既能辨认长期稳定的空间差异，也能直接比较单时点空间不均匀程度。

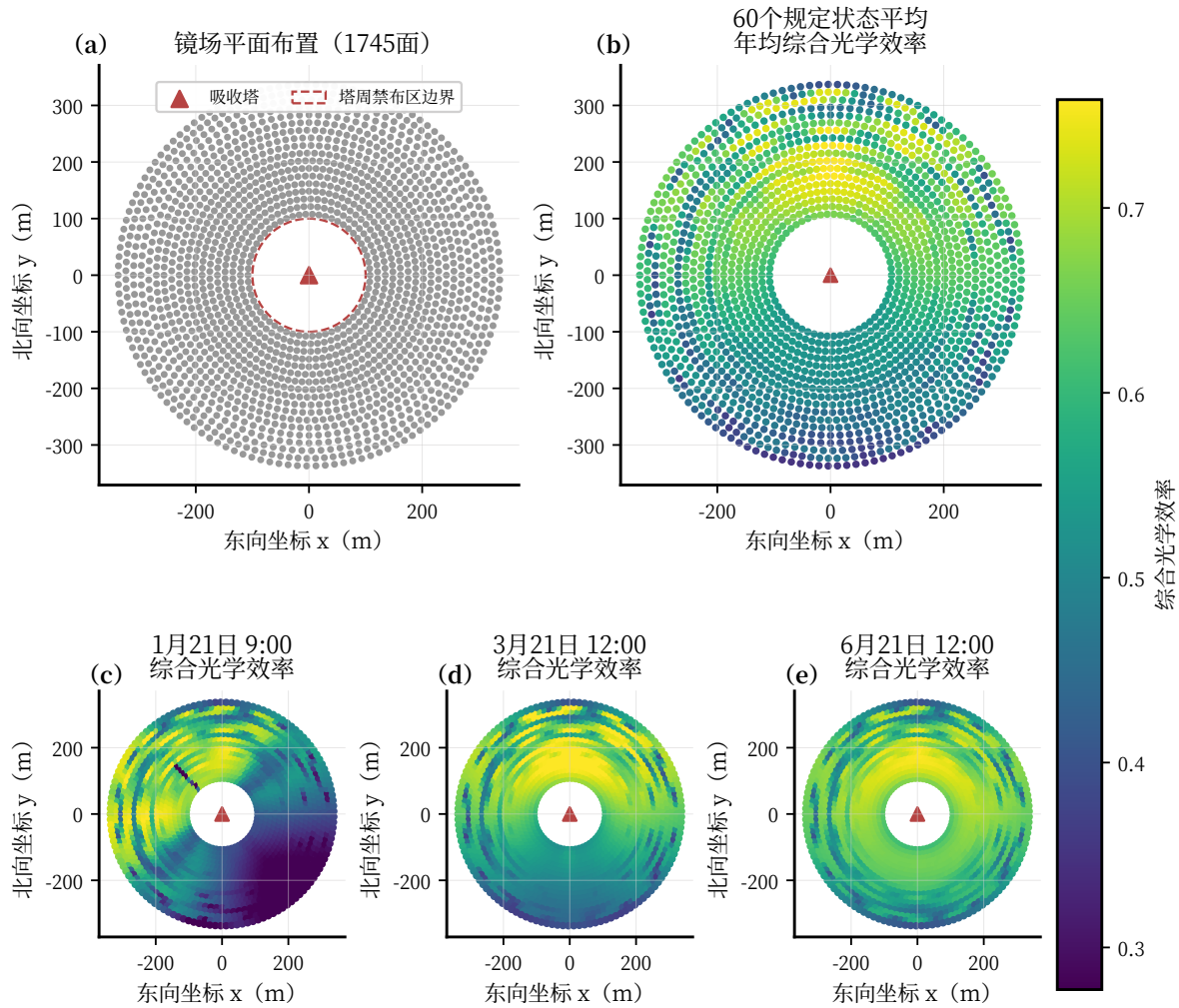


图 10 镜场布置、逐镜年均综合光学效率与三个代表状态的逐镜综合光学效率

图10(b) 给出逐镜年均综合光学效率，它把 60 个规定状态的短时方向性损失聚合为长期空间差异，可用于识别全年平均意义下持续高效或低效的镜位。图10(c) 表明低太阳高度角下效率具有明显的方向性不均匀，这是余弦方向、镜间遮挡与塔身阴影共同作用的结果；图10(d)、(e) 在正午状态下分布更接近环状，但仍随镜位变化。年均图比单时点更平滑，却没有退化为仅由半径决定的常数，说明必须逐镜、逐时点计算后再聚合，不能只按镜场半径给每面镜赋同一个平均效率。

月度功率同时受到 DNI、余弦效率、阴影遮挡效率、截断效率和大气透射率影响。因此，图11左侧同时展示主要效率分量，右侧展示最终单位面积功率，避免把季节变化错误归因于单一因素。

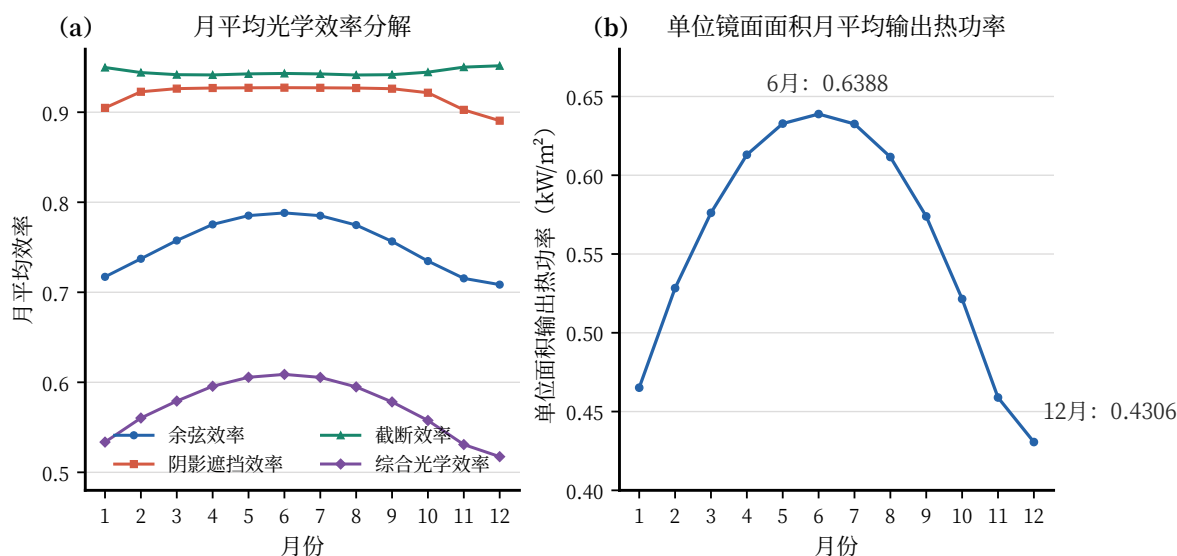


图 11 月平均光学效率分解与单位镜面面积输出热功率

图11显示，余弦效率和阴影遮挡效率由冬季向夏季上升，再向冬季下降；截断效率全年均高于 0.941，变化较小且冬季略高，但不能抵消冬季较低的 DNI、余弦效率和阴影遮挡效率。单位面积月平均输出热功率在 6 月达到最高值 0.6388 kW/m^2 ，在 12 月达到最低值 0.4306 kW/m^2 ，整体呈冬低夏高的规律。

5.11 题目要求的月度与年度结果

表3给出题目表 1 所需的 12 个月结果，效率和单位面积输出热功率均保留 4 位小数。

表 3 问题一每月平均光学效率及单位面积输出热功率

月	η	$\eta_{\cos}$	η_{sb}	η_{trunc}	P_A	月	η	$\eta_{\cos}$	η_{sb}	η_{trunc}	P_A
1	0.5336	0.7171	0.9048	0.9497	0.4652	7	0.6054	0.7851	0.9271	0.9425	0.6325
2	0.5603	0.7372	0.9227	0.9441	0.5283	8	0.5949	0.7747	0.9269	0.9414	0.6116
3	0.5792	0.7575	0.9262	0.9417	0.5761	9	0.5782	0.7565	0.9261	0.9417	0.5739
4	0.5956	0.7754	0.9269	0.9414	0.6130	10	0.5576	0.7347	0.9216	0.9445	0.5215
5	0.6056	0.7852	0.9271	0.9426	0.6328	11	0.5309	0.7154	0.9026	0.9501	0.4589
6	0.6088	0.7881	0.9273	0.9431	0.6388	12	0.5174	0.7085	0.8906	0.9516	0.4306

注： P_A 的单位为 kW/m^2 ，其余各列均为无量纲效率。

表4给出题目表 2 所需的年度指标。年平均镜场输出热功率为 34.9868 MW ；除以总镜面面积 62820 m^2 后，单位面积年平均输出热功率为 0.5569 kW/m^2 。两者由同一时点功率链条聚合，单位换算相互一致。

表 4 问题一题面离散年平均光学效率及输出热功率

指标	数值	单位
年平均综合光学效率	0.5723	—
年平均余弦效率	0.7529	—
年平均阴影遮挡效率	0.9192	—
年平均截断效率	0.9445	—
年平均镜场输出热功率	34.9868	MW
单位面积年平均输出热功率	0.5569	kW/m ²

5.12 本问结论与局限

本问建立了从太阳位置、理想镜面反射、有限矩形镜间相交、塔身圆柱阴影到有限圆柱接收的统一光学模型。其关键不是简单相乘各项平均效率，而是在同一镜面点和太阳盘方向上定义阴影、遮挡与接收事件，使 $\eta_{sb}\eta_{trunc}$ 严格等于未受阻挡且命中接收器的联合能量比例；随后逐镜、逐时点计算功率，再按题面权重聚合。

在题目规定的 60 个代表状态、1745 面理想平面镜、72 m 圆柱塔身和半角 4.65 mrad 均匀 pillbox 太阳盘下，年平均综合光学效率为 0.5723，年平均镜场输出热功率为 34.9868 MW，单位镜面面积年平均输出热功率为 0.5569 kW/m²。嵌套分辨率、四个固定种子、太阳盘半角和月份权重检查均表明结果较为稳定。

模型的限制也必须保留：60 状态离散平均不能替代真实逐日气象积分；理想镜面假设未包含面形、斜率、跟踪和污染损失；均匀 pillbox 未描述缺少参数依据的周日区；题面功率公式也没有包含接收器吸收与热损失。因此，本问结果适用于题面定义的光学评价与后续优化统一内核，不应直接外推为真实电站净发电量。

六、 问题二：统一规格定日镜场优化设计

6.1 优化目标与总体思路

问题二要求在镜面宽度、镜面高度和安装高度全场统一的条件下，同时确定吸收塔位置、定日镜规格、镜面总数以及所有镜心坐标。若直接把数千面镜的坐标全部作为独立变量，搜索维数过高，而且大部分随机组合会违反场界、禁布区或最小间距约束。因此，本文先构造能够批量产生合法镜位的参数化密排母池，再在该低维布局族内联合优化塔位、统一镜面规格与布局参数，最后使用问题一建立的完整光学评价器进行高精度复算。

设吸收塔平面坐标为 (x_T, y_T) ，统一镜面宽、高和安装高度分别为 w, h, z_H ，第 i 面

镜的镜心坐标为 (x_i, y_i, z_H) ，镜面总数为整数 N 。问题二的完整设计变量记为

$$\mathcal{D} = \{x_T, y_T, w, h, z_H, N, (x_i, y_i)_{i=1}^N\}.$$

对于题目给定的 60 个太阳状态，记第 k 个状态的法向直接辐射强度为 DNI_k ，第 i 面镜在该状态下由问题一统一光学模型算得的总光学效率为 $\eta_{ik}(\mathcal{D})$ 。则该状态的镜场输出热功率为

$$P_k(\mathcal{D}) = 10^{-3} \text{DNI}_k w h \sum_{i=1}^N \eta_{ik}(\mathcal{D}),$$

年平均输出热功率定义为

$$\bar{P}_{\text{field}}(\mathcal{D}) = \frac{1}{60} \sum_{k=1}^{60} P_k(\mathcal{D}).$$

由于问题二所有镜面规格相同，镜面总面积与单位面积年平均输出分别为

$$A_{\text{tot}} = N w h, \quad P_A(\mathcal{D}) = \frac{1000 \bar{P}_{\text{field}}(\mathcal{D})}{N w h}.$$

其中 10^{-3} 把逐状态功率由 kW 换算为 MW，1000 再将单位面积指标换算为 kW/m²，从而与题目要求的报告单位一致。因此，题目要求的优化目标不是总功率最大，而是在达到额定功率后最大化单位镜面面积的年平均输出。完整模型写为

$$\max_{\mathcal{D}} \quad \frac{1000 \bar{P}_{\text{field}}(\mathcal{D})}{N w h}, \quad \text{s.t.} \quad \bar{P}_{\text{field}}(\mathcal{D}) \geq 60 \text{ MW}.$$

这里不能单纯追求总功率：继续增加外圈镜面虽然可能提高 $\bar{P}_{\text{field}}$ ，却会扩大分母，并可能加重相邻镜面的阴影遮挡。故 60 MW 是决定候选能否被接受的容量硬约束， P_A 才是可行候选之间的排序目标。由于 N 为整数、镜位增删会改变遮挡邻接关系，且光线与镜面或接收器的相交状态具有切换边界，该模型是连续变量与离散变量耦合的非光滑约束优化问题。

6.2 几何约束与参数化布局

上述目标必须在全部物理与建设约束同时满足时才有意义。首先，吸收塔位于建设圆内，镜面规格、安装高度和镜面总数满足

$$x_T^2 + y_T^2 \leq 350^2, \quad 2 \leq h \leq w \leq 8, \quad 2 \leq z_H \leq 6, \quad N \in \mathbb{Z}_+.$$

定日镜绕其水平轴跟踪太阳。按照本文镜高方向与旋转轴垂直的定义，任意姿态下镜面最低点的保守包络由半镜高控制，因此严格离地约束为

$$z_H - \frac{h}{2} > 0.$$

该式既检查题面给出的安装高度范围，也防止优化器通过令镜面边缘接触地面获得不可实施的面积优势。

建设场地为以原点为圆心、半径 350 m 的圆形区域。场界按镜心口径检查，故对每一面镜均有

$$x_i^2 + y_i^2 \leq 350^2, \quad i = 1, \dots, N.$$

塔周 100 m 禁布区随塔位共同移动，而不是固定在建设区域中心，因此

$$(x_i - x_T)^2 + (y_i - y_T)^2 \geq 100^2, \quad i = 1, \dots, N.$$

镜面有限尺寸已经在阴影、遮挡和截断计算中体现；上述两个不等式仅规定镜心是否位于合法建设区域。任意两面不同定日镜的底座中心距离须至少比统一镜宽多 5 m，即

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \geq w + 5, \quad 1 \leq i < j \leq N.$$

程序使用空间索引找出所有可能违反该式的近邻镜对，再逐对进行确定性检查，因而“相邻”不是预先固定的邻接关系，也不会分区边界处漏检。

综上，问题二实际求解的约束优化模型可集中表示为

$$\begin{aligned} \max_{\mathcal{D}} \quad & P_A(\mathcal{D}) = \frac{1000 \bar{P}_{\text{field}}(\mathcal{D})}{Nwh}, \\ \text{s.t.} \quad & \bar{P}_{\text{field}}(\mathcal{D}) \geq 60 \text{ MW}, \\ & x_T^2 + y_T^2 \leq 350^2, \\ & x_i^2 + y_i^2 \leq 350^2, \quad i = 1, \dots, N, \\ & (x_i - x_T)^2 + (y_i - y_T)^2 \geq 100^2, \quad i = 1, \dots, N, \\ & (x_i - x_j)^2 + (y_i - y_j)^2 \geq (w + 5)^2, \quad 1 \leq i < j \leq N, \\ & 2 \leq h \leq w \leq 8, \quad 2 \leq z_H \leq 6, \quad z_H - h/2 > 0, \\ & N \in \mathbb{Z}_+. \end{aligned}$$

在最终资格判定中，功率约束不仅要求四种子池化结果达到 60 MW，还要求四个固定种子的细分分辨率结果分别达到 60 MW；后者是对数值采样不确定性的保守可行性检查，不另行改变目标函数。

为在给定最小间距下提高候选容量，母池采用三角密排点阵，最近邻间距直接取 $d = w + 5$ ，并以吸收塔为圆心设置内、中、外三个径向区。搜索变量除 w, h, z_H, y_T 外，还包括两条径向边界、各区点阵相位和分区过渡带宽度；塔横坐标在该参数化布局中固定为 $x_T = 0$ 。生成候选后，对全部镜面统一执行场界、禁布区和全局间距检查，跨分区镜对同样参与检查，因此分区拼接不会绕过几何硬约束。

搜索阶段曾允许不同径向区域采用不同点阵相位，但最终得到 $\phi_1 = \phi_2 = \phi_3 = 0$ ，没有形成具有实际差异的分区排列。因此，最终方案按统一三角密排表述，不再人为划分内、中、外区。图12给出最终镜场平面布局，并标出建设场界和随塔位移动的 100 m 禁布区，用于直观核对镜位与几何约束的关系。

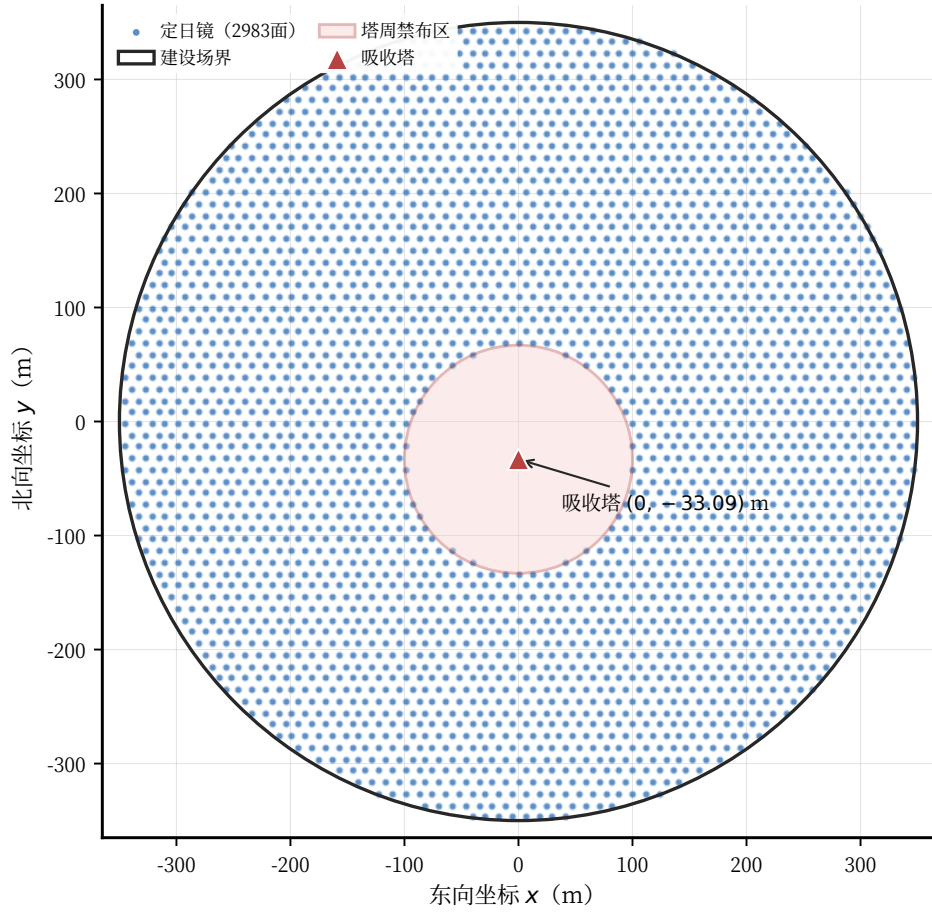


图 12 问题二最终定日镜场平面布局与几何边界

6.3 光学评价与粒子群搜索

每一个合法候选均调用问题一的三维光学模型。对于 60 个规定太阳状态，先由太阳高度角和方位角得到入射方向，再令每面镜的法向平分指向太阳与指向接收器中心的两个单位向量，使中心光线反射至接收器。评价同时考虑余弦损失、大气透射、镜间阴影遮挡、塔身入射阴影以及有限圆柱接收器的截断损失。镜面点与太阳盘方向构成联合 Sobol 样本，同一条样本光线依次判断入射可见、出射无遮挡和接收器命中，从而避免对同一能量重复扣减。

候选生成包含镜面增删、边界裁剪和遮挡关系切换，目标函数并不光滑，故采用粒子群算法。每个粒子编码为

$$\mathbf{x} = (w, h/w, c, y_T, r_1, r_2, \phi_1, \phi_2, \phi_3, b),$$

其中 c 为离地裕量参数， r_1, r_2 为分区边界， ϕ_1, ϕ_2, ϕ_3 为三区相位， b 为过渡带参数。候选若违反任一几何硬约束，则在进入光学评价前淘汰。

求解采用三级精度：首先用代表性太阳状态进行粗筛，只删除明显劣解；其次将入围候选全部送入 60 状态评价并重新排序；最后对前列候选使用 16 个镜面点乘 8 个太

阳方向和 32 个镜面点乘 16 个太阳方向的两级嵌套分辨率，并在随机种子 2023、2024、2025 和 2026 下分别复算。正式值取细分分辨率四种子联合事件计数的池化结果，同时要求四个单种子功率均不低于 60 MW。

搜索允许矩形镜面和不同径向区域采用不同点阵相位。最终粒子收敛到 $h/w = 1$ 且 $\phi_1 = \phi_2 = \phi_3 = 0$ ，说明在当前目标、约束和搜索预算下，矩形宽高比与差异化相位没有进一步改善最终指标。因此，最终方案采用统一相位的三角密排结构。

6.4 优化结果

最终吸收塔坐标为 $(0, -33.0924)$ m，全部定日镜采用 6.7083×6.7083 m 统一规格，安装高度为 3.8926 m，共布置 2983 面镜，总镜面面积为 134238.7113 m^2 。最小镜心间距为 11.7083 m，全部镜面均位于建设场界内、塔周禁布区外，完整旋转包络的最小离地净空为 0.5385 m。

表 5 问题二各月平均光学效率及输出热功率

月份	光学效率	余弦效率	阴影遮挡效率	截断效率	单位面积功率/(kW/m ²)
1	0.4178	0.7433	0.7118	0.9396	0.3649
2	0.4547	0.7587	0.7577	0.9274	0.4288
3	0.4714	0.7733	0.7708	0.9181	0.4689
4	0.4822	0.7847	0.7716	0.9154	0.4962
5	0.4883	0.7897	0.7720	0.9148	0.5102
6	0.4903	0.7908	0.7723	0.9151	0.5144
7	0.4882	0.7896	0.7720	0.9149	0.5100
8	0.4819	0.7843	0.7716	0.9156	0.4953
9	0.4709	0.7726	0.7706	0.9185	0.4673
10	0.4511	0.7568	0.7536	0.9287	0.4220
11	0.4136	0.7420	0.7064	0.9404	0.3583
12	0.3945	0.7365	0.6823	0.9427	0.3295

为观察不同季节的损失来源，图13将表5中的效率分量和单位面积功率转化为月度曲线。冬季太阳高度较低时，阴影遮挡效率下降最明显；随太阳高度升高，单位面积输出在年中达到较高水平。图中的变化只对应题目规定的 60 个晴空太阳状态。

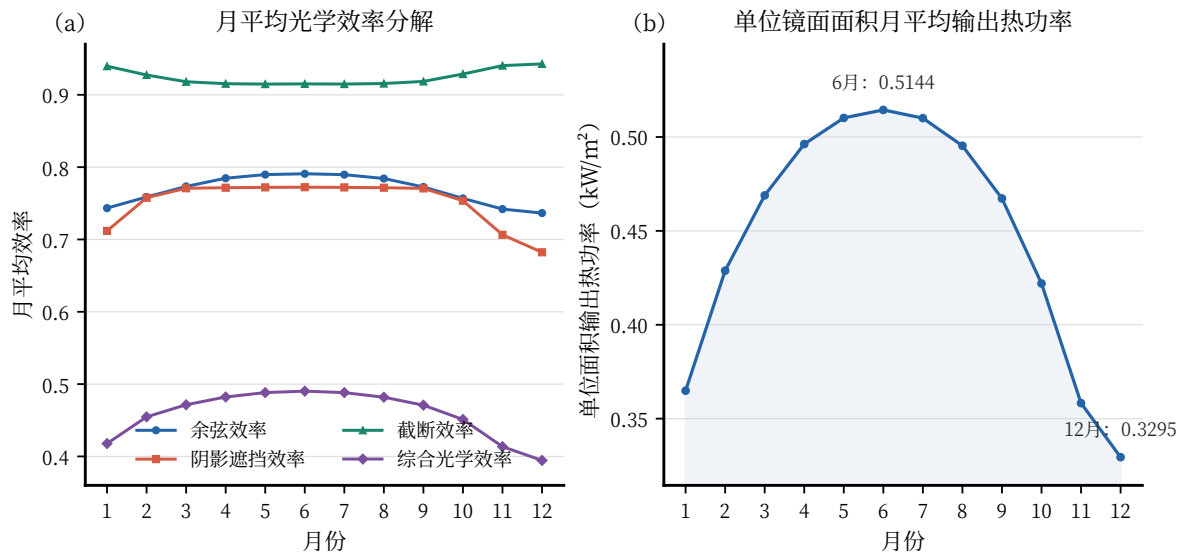


图 13 问题二月平均光学效率分解与单位面积输出热功率

表 6 问题二年平均效率与输出热功率

光学效率	余弦效率	阴影遮挡效率	截断效率	输出热功率/MW	单位面积功率/(kW/m ²)
0.4587	0.7685	0.7511	0.9243	60.0257	0.4472

6.5 收敛性与结果边界

从 16×8 到 32×16 分辨率，年平均功率相对变化为 0.0109%，12 个月单位面积功率的最大相对变化为 0.0824%，分别低于预设的 0.3% 和 0.5% 阈值。四个细分分辨率种子的年平均功率依次为 60.0037、60.0294、60.0272 和 60.0424 MW，标准差为 0.0140 MW，四个结果均满足 60 MW。

正式池化结果相对 60 MW 的裕量仅为 0.0257 MW，最小种子裕量仅为 0.0037 MW，

表 7 问题二最终设计参数

参数	最终值
吸收塔坐标	(0, -33.0924) m
定日镜统一尺寸	6.7083 × 6.7083 m
定日镜安装高度	3.8926 m
定日镜总数	2983 面
定日镜总面积	134238.7113 m ²
最小镜心间距	11.7083 m

因此该方案属于高单位面积效率的约束边界解，而不是具有较大容量冗余的工程保守解。本文结论仅说明该方案在既定理想平面镜、塔身圆柱、有限圆柱接收器和数值分辨率口径下满足题目约束；不声称它是所有可能逐镜自由布局中的全局最优解。全部 2983 面镜的坐标与统一规格见提交文件 `result2.xlsx`。

七、问题三：可变规格定日镜场优化设计

7.1 问题扩展与优化目标

问题二要求全场镜面规格统一，问题三则允许每面定日镜分别选择宽度、高度和安装高度，并允许重新确定吸收塔位置、镜面总数和全部镜位。新增自由度可以缩小低贡献镜面以减少总面积，也可以调整安装高度以改善遮挡关系；但镜面尺寸一旦变化，最小间距、镜面面积和逐镜光学效率会同时变化。因此，本问不能在问题二镜场上只替换少量规格，而应把布局生成与逐镜规格优化统一起来。

设吸收塔平面坐标为 (x_T, y_T) ，第 i 面定日镜的镜心为 (x_i, y_i, z_i) ，宽度、高度和面积分别为 w_i, h_i 和 $a_i = w_i h_i$ ，镜面总数为 N 。第 k 个规定太阳状态的法向直接辐射强度为 D_k ，问题一光学模型计算的第 i 面镜总光学效率为 η_{ik} 。该时刻镜场输出热功率为

$$P_k = 10^{-3} D_k \sum_{i=1}^N a_i \eta_{ik}. \quad (38)$$

式中的 10^{-3} 将 DNI 与镜面面积相乘所得的 kW 换算为 MW。对题目规定的 12 个月、每月 5 个时刻作等权平均，得到年平均输出热功率

$$\bar{P} = \frac{1}{60} \sum_{k=1}^{60} P_k. \quad (39)$$

总镜面面积和单位面积年平均输出热功率为

$$A_{\text{tot}} = \sum_{i=1}^N w_i h_i, \quad P_A = \frac{1000 \bar{P}}{A_{\text{tot}}} \quad (\text{kW/m}^2). \quad (40)$$

式(40)中的 1000 用于将 MW 换算为 kW。由此，本问的目标不是无约束地增加总功率，而是在年平均功率达到 60 MW 后最大化 P_A ：

$$\max_{\mathcal{D}_3} P_A, \quad \text{s.t.} \quad \bar{P} \geq 60 \text{ MW}, \quad (41)$$

其中

$$\mathcal{D}_3 = \{x_T, y_T, N, (x_i, y_i, z_i, w_i, h_i)_{i=1}^N\}.$$

容量约束负责筛除总面积过小的高效率方案， P_A 负责比较达到容量要求后的镜场。二者次序不能互换，否则优化器会偏向面积很小但总功率不足的镜场。

当各镜面积不同时，镜场平均效率必须与式(38)保持一致。对余弦、阴影遮挡、截断等任一效率分量 q ，本文采用面积加权

$$\bar{\eta}_{q,k} = \frac{\sum_{i=1}^N a_i \eta_{q,ik}}{A_{\text{tot}}}. \quad (42)$$

若使用逐镜等权，小尺寸镜面会获得与大尺寸镜面相同的权重，所得平均效率将不能还原实际功率，故不采用该口径。

7.2 逐镜几何约束

吸收塔和镜心须位于半径 350 m 的建设圆内，镜心须处于以吸收塔为圆心、半径 100 m 的禁布区外，即

$$x_T^2 + y_T^2 \leq 350^2, \quad x_i^2 + y_i^2 \leq 350^2, \quad (x_i - x_T)^2 + (y_i - y_T)^2 \geq 100^2. \quad (43)$$

第 i 、 j 面镜的宽度不同时，镜心间距采用两镜宽度半和加 5 m：

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \geq \frac{w_i + w_j}{2} + 5, \text{ 当 } i \neq j. \quad (44)$$

该式在 $w_i = w_j$ 时退化为问题二的 $d_{ij} \geq w + 5$ ，因而两问口径一致。程序利用空间索引搜索所有可能冲突的近邻对，再按式(44)逐对检查，分区边界上的镜对也包含在内。

根据题目边界和镜面宽度一般不小于高度的要求，每面镜满足

$$2 \leq h_i \leq w_i \leq 8, \text{ 当 } i \neq j. \quad (45)$$

为保证镜面转动时不接触地面，以半镜高构造保守旋转包络：

$$z_i - \frac{h_i}{2} > 0. \quad (46)$$

综上，式(41)与式(43)–(46)构成连续规格、离散镜数和非光滑光学效率耦合的约束优化模型。

7.3 连续布局生成与遗传搜索

若把 2939 面左右定日镜的全部镜位和规格直接编码，变量数接近 $5N$ ，有限预算内难以形成有效群体搜索。因此，第一层不直接移动每个镜心，而是用少量连续参数生成完整合法镜场。考虑圆形场界和密排需求，采用两径向区三角晶格。内、外区可以具有不同基础尺寸、相位和高宽比，所有候选均从空场重新生成，不继承问题二坐标。

遗传算法个体编码为

$$\theta = (y_T, R_b, w_{\text{in}}, w_{\text{out}}, \phi_{\text{in}}, \phi_{\text{out}}, \rho_{\text{in}}, \rho_{\text{out}}, c), \quad (47)$$

其中 R_b 为两区径向分界, $\rho = h/w$ 为高宽比, c 为离地净空参数。 ϕ 表示三角点阵的**纵向平移相位**, 不是点阵旋转角。具体地, 对内区或外区 $q \in \{\text{in}, \text{out}\}$, 令最近邻间距和相邻行距分别为

$$d_q = w_q + 5, \quad \Delta y_q = \frac{\sqrt{3}}{2} d_q, \quad (48)$$

则该区三角点阵第 m 行、第 n 列的母池坐标可写为

$$y_{q,m} = y_T + (m + \phi_q) \Delta y_q, \quad x_{q,mn} = \left[n + \frac{1}{2}(m \bmod 2) \right] d_q. \quad (49)$$

因此, $\phi_q \in [0, 1)$ 只使该区全部点阵行沿 y 方向平移 $\phi_q \Delta y_q$, 不改变最近邻距离、交错结构和点阵方向。例如, $\phi_q = 0.5$ 表示整体平移半个行距。引入该变量是为了调整点阵行与塔周禁布圆、径向分区边界及另一分区点阵的相对位置, 从而允许镜面数量和合法镜位随布局连续变化。连续搜索范围为

$$\begin{aligned} y_T &\in [-120, 20], \quad R_b \in [120, 320], \\ w_{\text{in}}, w_{\text{out}} &\in [2, 8], \\ \phi_{\text{in}}, \phi_{\text{out}} &\in [0, 0.99], \\ \rho_{\text{in}}, \rho_{\text{out}} &\in [0.85, 1], \quad c \in [0.5, 0.9]. \end{aligned} \quad (50)$$

上述变量全部采用浮点实数编码。初始化时设置少量边界覆盖点, 是为了避免初始种群遗漏 2 m 和 8 m 附近区域; 交叉和变异仍在式(50)的整个连续区间内进行, 并非只在若干离散尺寸中选择。

对每个个体, 先按两区参数生成候选三角晶格点, 再依次执行场界、禁布区和异尺寸间距检查, 只有合法镜面才被接纳。镜面总数 N 由基础尺寸、区界、相位、塔位和场界共同产生, 故随个体自然变化。本轮 14 个体迭代 5 代, 共完成 70 次布局评价, 实际镜数覆盖 2498 至 7128 面, 说明搜索确实改变了镜面数量与位置。

候选评价采用容量优先的约束排序。未达到安全功率阈值时, 优先提高 $\bar{P}$; 达到阈值后, 再按 P_A 排序。搜索层使用覆盖四季和不同太阳高度的代表状态进行快速淘汰, 前列非重复候选再用全部 60 状态和 8×4 光线样本复核。

7.4 逐镜连续细化与容量修复

外层搜索得到的候选布局含 2939 面镜, 吸收塔纵坐标约为 -37.43 m。随后固定镜心坐标, 先连续优化安装高度, 再计算每面镜的年均单位面积光学贡献。低贡献镜面优先进入规格调整集合, 由差分进化搜索受调镜比例、贡献排序指数和宽高缩减尺度; 最终变化量落实到每面镜的连续 w_i, h_i, z_i 。每次接受新阶段前均重新检查式(43)–(46), 并要求单位面积功率提高且中精度总功率保持在容量安全带内。

逐镜细化的前三阶段分别得到 60.3442、60.2211 和 60.1485 MW 的单种子 8×4 功率, 单位面积功率依次提高到 0.4532、0.4555 和 0.4568 kW/m²。然而, 第三阶段在 16×8 、

32 × 16 和四种子正式复核下只有 59.9052 MW。该结果说明：当候选逼近 60 MW 边界时，中精度采样的乐观误差足以改变可行性，不能因单位面积指标更高而保留该方案。

为在相同镜数和镜位上恢复正式容量裕量，设第一、第二连续细化阶段的逐镜参数分别为 $\boldsymbol{v}_i^{(1)} = (z_i^{(1)}, w_i^{(1)}, h_i^{(1)})$ 和 $\boldsymbol{v}_i^{(2)}$ ，采用

$$\boldsymbol{v}_i = (1 - \alpha)\boldsymbol{v}_i^{(1)} + \alpha\boldsymbol{v}_i^{(2)}, \quad \alpha = 0.2. \tag{51}$$

式(51)在两组合法连续规格之间插值，并不把镜面恢复为少数标准尺寸。它适度增加总面积，以牺牲部分 P_A 换取正式功率裕量。插值方案经完整复核后满足全部约束，因此作为最终方案。

7.5 优化结果

最终吸收塔坐标为 (0, −37.4265) m，共布置 2939 面定日镜，总镜面面积 132952.1202 m²。镜宽为 4.7554–6.8000 m，镜高为 4.7482–6.8000 m，安装高度为 3.4091–4.8225 m。六位小数口径下分别有 205 种镜宽、205 种镜高和 2512 种安装高度，说明最终结果保留了逐镜连续规格，而非退化为两三种离散镜面。

表 8 问题三各月平均光学效率及输出热功率

月	η	$\eta_{\cos}$	η_{sb}	η_{trunc}	P_A	月	η	$\eta_{\cos}$	η_{sb}	η_{trunc}	P_A
1	0.4228	0.7458	0.7185	0.9375	0.3693	7	0.4934	0.7904	0.7818	0.9126	0.5155
2	0.4598	0.7608	0.7655	0.9253	0.4337	8	0.4870	0.7854	0.7809	0.9133	0.5006
3	0.4766	0.7749	0.7795	0.9159	0.4741	9	0.4761	0.7742	0.7793	0.9163	0.4724
4	0.4873	0.7858	0.7809	0.9131	0.5015	10	0.4562	0.7589	0.7613	0.9266	0.4268
5	0.4935	0.7904	0.7819	0.9126	0.5157	11	0.4185	0.7444	0.7130	0.9383	0.3626
6	0.4955	0.7915	0.7823	0.9128	0.5199	12	0.3993	0.7390	0.6882	0.9406	0.3335

注： P_A 的单位为 kW/m²，其余各列均为无量纲效率。

表8给出题目表 1 所需的 12 个月结果。冬季太阳高度较低，阴影遮挡效率下降，单位面积功率相应较小；5–7 月太阳高度较高，阴影遮挡损失减弱，单位面积功率达到全年较高水平。该解释只描述题目 60 个规定状态下的季节变化，不外推到未计算的逐日运行条件。

表 9 问题三年平均效率与输出热功率

指标	数值	指标	数值	指标	数值
光学效率	0.4639	余弦效率	0.7701	阴影遮挡效率	0.7594
截断效率	0.9221	输出热功率	60.1119 MW	单位面积功率	0.4521 kW/m ²

表 10 问题三最终设计参数与连续规格范围

参数	最终值	参数	最终值
吸收塔坐标	(0, -37.4265) m	定日镜总数	2939 面
定日镜总面积	132952.1202 m ²	最小离地净空	0.0500 m
镜面宽度范围	4.7554–6.8000 m	镜面高度范围	4.7482–6.8000 m
安装高度范围	3.4091–4.8225 m	连续规格种类	205/205/2512 种 *

注：* 依次为六位小数口径下的镜宽、镜高和安装高度种类数。

与问题二同一光学和数值口径下的统一规格方案相比，问题三单位面积功率由 0.447156 kW/m² 提高到 0.452132 kW/m²，相对提高 1.1127%；总面积减少约 1286.59 m²，而正式年平均功率仍达到 60.111866 MW。该比较支持异质规格在已搜索布局族中能够削减低贡献面积并维持容量，但不等同于对所有可能自由布局的全局最优证明。

7.6 正式复核与结果边界

正式评价使用全部 60 个太阳状态。镜面位置与太阳盘方向的联合采样先采用 16 × 8 分辨率，再提升至 32 × 16；年平均功率相对变化为 0.0450%，12 个月单位面积功率的最大相对变化为 0.1413%，数值收敛判定通过。

在种子 2023、2024、2025 和 2026 下，细分分辨率年平均功率依次为 60.0805、60.1070、60.1260 和 60.1340 MW，标准差为 0.0206 MW，四个结果均满足 60 MW。全场 2939 面镜均位于建设场界内和禁布区外，异尺寸间距违规数为 0， $h_i > w_i$ 违规数为 0，最小离地净空为 0.05 m。

最终池化功率相对 60 MW 的裕量为 0.1119 MW，最小种子裕量为 0.0805 MW。该裕量足以通过本文的数值采样检验，但尚未包含镜面制造误差、跟踪误差、地形起伏和长期污染等工程不确定性。因此，本文把该方案表述为给定物理模型、连续参数化布局族和记录搜索预算下的最佳已验证可行方案，不宣称其为全部逐镜自由设计中的全局最优。全部镜面坐标、宽度、高度和安装高度见提交文件result3.xlsx。

八、模型分析与评价

8.1 模型的整体结构

三个问题共用同一套太阳位置、镜面跟踪、光线反射和损失计算模型，区别只在于待求变量逐步增加。问题一在给定镜场上完成光学性能评价，为后续优化提供统一的目标函数与约束判定器；问题二在统一镜面规格条件下联合搜索塔位、布局 and 镜面尺寸；问题三进一步开放逐镜宽度、高度和安装高度，并以面积加权方式计算异质镜场功率。

这样的递进结构保证了三问数值能够在同一物理口径下比较，也避免在优化阶段使用与评价阶段不一致的近似指标。

8.2 模型优点

首先，模型保持了从太阳矢量到接收器命中的完整几何链条。每面定日镜均依据入射方向和指向接收器中心的方向实时确定法向量，余弦损失、阴影遮挡、大气透射和截断损失均在同一光线样本上计算，因此各效率分量与最终功率之间具有明确对应关系。对于问题三的异质镜面，采用镜面面积加权而非逐镜等权，使效率汇总能够严格还原功率计算式。

其次，优化过程把容量约束置于单位面积功率比较之前。问题二和问题三均先要求年平均输出热功率不低于 60 MW，再最大化单位镜面面积年平均输出热功率，从而排除了面积很小、效率较高但不能完成供热任务的伪优方案。场界、禁布区、异尺寸间距、宽高关系及离地净空均在候选生成和正式复核阶段再次检查，使所得方案同时满足光学目标和几何可实施条件。

最后，模型采用分层精度控制搜索误差。低分辨率用于大范围筛选，高分辨率和多随机种子只用于候选复核。问题一的分辨率与随机种子检验表明评价结果较为稳定；问题二、问题三的最终方案也均经过两级分辨率和四种子检验。问题三细化过程中还主动剔除了在高分辨率下跌破 60 MW 的候选，说明容量判定以正式复核结果为准，而不是以搜索阶段的乐观估计为准。

8.3 模型局限与改进方向

本文结论适用于题目给定的 60 个离散太阳状态及所建立的理想化光学模型。太阳盘采用均匀 pillbox 角分布，尚未引入周日亮度、镜面斜率误差、跟踪误差和长期积灰；接收器按几何命中统计截断效率，也未进一步模拟接收器表面热流密度与热损失。因此，所得功率表示规定状态下的光学输出，不应直接解释为包含天气和设备可用率的实际电站年发电量。

此外，问题二和问题三含有离散镜数、非光滑遮挡关系及大量连续变量，粒子群、遗传算法与差分进化给出的是记录搜索范围和预算下的最佳已验证可行解，而非数学意义上的全局最优解。特别是两问的容量裕量均较小，工程设计时还需为制造、跟踪和气象不确定性预留更大的安全裕度。后续可引入实测 DNI 权重、非均匀太阳形状和误差卷积，并把接收器峰值热流及容量裕量纳入稳健多目标优化，以提高工程适用性。

九、总结

本文建立了由太阳位置、双向量定日镜跟踪、光线级阴影遮挡与截断判定组成的统一光学模型，并据此完成给定镜场评价、统一规格镜场优化和逐镜连续规格优化。

对于问题一，给定镜场在题目 60 个太阳状态下的年平均光学效率为 0.5723，年平均输出热功率为 34.9868 MW，单位镜面面积年平均输出热功率为 0.5569 kW/m²。对于问题二，最密三角晶格与分区参数化搜索得到满足 60 MW 约束的统一规格方案，其年平均输出热功率为 60.0257 MW，单位面积功率为 0.4472 kW/m²。对于问题三，从空场重新生成布局并连续调整逐镜宽度、高度和安装高度后，最终 2939 面定日镜的总面积为 132952.1202 m²，年平均输出热功率为 60.1119 MW，单位面积功率达到 0.4521 kW/m²，较问题二提高 1.1127%。

综合三问可见，在相同光学评价口径下，参数化密排能够控制大规模镜位搜索的计算量，逐镜连续规格则能进一步削减低贡献镜面面积。最终结果均通过分辨率和多随机种子复核，但其有效范围仍受离散太阳状态、理想化太阳盘以及未计入工程误差等假设限制，因而应将其理解为本文模型与搜索预算下的可行设计结果。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2023 高教社杯全国大学生数学建模竞赛 A 题：定日镜场的优化设计及附件 [Z]. 2023.
- [2] latexstudio. CUMCMThesis: 全国大学生数学建模竞赛 $\text{\LaTeX}$ 论文模板 [EB/OL]. <https://github.com/latexstudio/CUMCMThesis>.
- [3] Wang Y, Potter D, Asselineau C A, et al. Verification of optical modelling of sunshape and surface slope error for concentrating solar power systems[J]. Solar Energy, 2020, 195: 461–474. doi:10.1016/j.solener.2019.11.035.

A 附录：程序

1.1 问题一程序

```
1  #-*- coding: utf-8 -*-
2  """
3  2023年高教社杯全国大学生数学建模竞赛 A题 问题1 —— 定日镜场的优化设计
4
5  计算内容：
6  1. 每月21日（12个月）当地时间 9:00 / 10:30 / 12:00 / 13:30 / 15:00 五个时点的
7     平均光学效率、平均余弦效率、平均阴影遮挡效率、平均截断效率、
8     单位面积镜面平均输出热功率（对应题目表 1）；
9  2. 上述指标的年平均值与年平均输出热功率（对应题目表 2）。
10
11  主要修正（相对早期版本）：
12  - 吸收塔高度按题意取 80 m（集热器中心离地高度），而不是 84 m；
13  - 太阳赤纬按“以春分 3 月 21 日为第 0 天起算”的天数 D 计算，
14     每月 21 日对应 D = -59, -28, 0, 31, 61, 92, 122, 153, 184, 214, 245, 275；
15  - 太阳方向矢量使用题目附录的太阳方位角  $\gamma$ （分上午/下午两个象限），
16     不再用纬度  $\varphi$  近似；
17  - 阴影/遮挡判定要求射线交点位于射线前进方向（ $t > 0$ ），避免把
18     位于反方向的相邻镜子误判为遮挡；
19  - 截断效率不再取常数 0.8，而是按太阳光锥半顶角 4.65 mrad 计算反射光斑
20     与集热器（直径 7 m）的相交面积占比，且只统计未被阴影遮挡的网格点。
21  """
22  import time
23
24  import numpy as np
25  import pandas as pd
26  from pathlib import Path
27
28  # 基础参数（题意给定）
29  H_MIRROR = 4.0 # 定日镜安装高度（镜面中心离地高度），m
30  W_MIRROR = 6.0 # 镜面宽度（左右两边之间距离），m
31  H_MIRROR_SIZE = 6.0 # 镜面高度（上下两边之间距离），m
32  H_HEAT = 80.0 # 吸收塔高度（集热器中心离地高度），m
33  R_REC = 3.5 # 集热器半径（圆柱直径 7 m），m
34  THETA_SUN = 4.65e-3 # 太阳光锥半顶角，rad
35  PHI = 39.4 * np.pi / 180.0 # 当地纬度（北纬为正）
36  H_SEA = 3.0 # 海拔高度，km
37  G0 = 1.366 # 太阳常数，kW/m^2
38  REF_EFF = 0.92 # 镜面反射率
39  GROUND_VEC = np.array([0.0, 0.0, -1.0]) # 地面法向（向下）
40  N_NEAREST = 6 # 阴影/遮挡只考察最近 6 面相邻定日镜
41  N_W = 20 # 镜面宽度方向网格数
42  N_D = 20 # 镜面高度方向网格数
43
44  # 每月 21 日以春分（3 月 21 日，一年中第 80 天）为第 0 天起算的天数 D
45  DAYS_D = [-59, -28, 0, 31, 61, 92, 122, 153, 184, 214, 245, 275]
46  MONTHS = [f'{m}月21日' for m in range(1, 13)]
47  STS = [9.0, 10.5, 12.0, 13.5, 15.0] # 当地时间，h
48
49
50  # 太阳位置与辐照度
51  def calc_basic_data(day_D, st, phi):
52      """太阳时角  $\omega$ 、赤纬角  $\delta$ 、高度角  $\alpha$ 、方位角  $\gamma$ （题目附录公式）"""
53      omega = np.pi / 12.0 * (st - 12.0)
54      delta = np.arcsin(np.sin(2.0 * np.pi * day_D / 365.0) *
55                      np.sin(2.0 * np.pi * 23.45 / 360.0))
56      sin_alpha = (np.cos(delta) * np.cos(phi) * np.cos(omega) +
```

```

57     np.sin(delta) * np.sin(phi))
58     alpha = np.arcsin(np.clip(sin_alpha, -1.0, 1.0))
59     cos_gamma = ((np.sin(delta) - np.sin(alpha) * np.sin(phi)) /
60                 (np.cos(alpha) * np.cos(phi)))
61     cos_gamma = np.clip(cos_gamma, -1.0, 1.0)
62     gamma = np.arccos(cos_gamma)
63     if omega > 0.0: # 下午太阳偏西
64         gamma = 2.0 * np.pi - gamma
65     return omega, delta, alpha, gamma
66
67
68 def sun_vector(alpha, gamma):
69     """太阳方向单位矢量 (从定日镜指向太阳), x 东、y 北、z 上"""
70     return np.array([np.cos(alpha) * np.sin(gamma),
71                     np.cos(alpha) * np.cos(gamma),
72                     np.sin(alpha)])
73
74
75 def calc_DNI(h_sea, alpha, G0):
76     """法向直接辐射照度, kW/m^2 (题目附录公式)"""
77     a = 0.4237 - 0.00821 * (6.0 - h_sea) ** 2
78     b = 0.5055 + 0.00595 * (6.5 - h_sea) ** 2
79     c = 0.2711 + 0.01858 * (2.5 - h_sea) ** 2
80     return G0 * (a + b * np.exp(-c / np.sin(alpha)))
81
82
83 # 镜面几何
84 def calc_mirror_geometry(x, y, u):
85     """镜面中心 base、反射方向 v、法向 n 及镜面局部坐标轴 axis_x / axis_y"""
86     base = np.column_stack([x, y, np.full_like(x, H_MIRROR)])
87     tower = np.array([0.0, 0.0, H_HEAT])
88     v = tower - base
89     v = v / np.linalg.norm(v, axis=1, keepdims=True)
90     # 法向取太阳方向与反射方向的角平分方向, 满足反射定律
91     n = u + v
92     n = n / np.linalg.norm(n, axis=1, keepdims=True)
93     axis_x = np.cross(n, GROUND_VEC)
94     axis_x = axis_x / np.linalg.norm(axis_x, axis=1, keepdims=True)
95     axis_y = np.cross(n, axis_x)
96     axis_y = axis_y / np.linalg.norm(axis_y, axis=1, keepdims=True)
97     return base, v, n, axis_x, axis_y
98
99
100 # 阴影遮挡效率
101 def calc_block_eff(base, v, n, axis_x, axis_y, u, neighbors):
102     """
103     阴影遮挡效率: 把每面镜面划分为 N_W x N_D 个网格点,
104     对每个网格点分别做“指向太阳”( +u, 入射阴影)与“指向集热器”( +v, 反射遮挡)
105     的射线, 判断射线与最近 6 面相邻镜面的交点是否落在该镜面矩形内。
106     交点必须位于射线前进方向 (t > 0)。
107     """
108     n_mirror = len(base)
109     mesh_p, mesh_q = np.meshgrid(np.arange(N_W), np.arange(N_D))
110     a_off = (mesh_p - (N_W - 1) / 2.0) * (W_MIRROR / N_W) # (N_D, N_W)
111     b_off = (mesh_q - (N_D - 1) / 2.0) * (H_MIRROR_SIZE / N_D)
112     offsets = (a_off[None, :, :, None] * axis_x[:, None, None, :] +
113              b_off[None, :, :, None] * axis_y[:, None, None, :])
114     little = base[:, None, None, :] + offsets # (N, N_D, N_W, 3)
115
116     block_num = np.zeros(n_mirror, dtype=int)
117     blocked_grid = np.zeros((n_mirror, N_D, N_W), dtype=bool)
118
119     for i in range(n_mirror):
120         ind = neighbors[i]
121         diff = base[ind, None, None, :] - little[i] # (K, N_D, N_W, 3)
122         n_j = n[ind[:, None, None, :]] # (K, 1, 1, 3)
123         with np.errstate(divide='ignore', invalid='ignore'):
124             t_out = np.sum(n_j * diff, axis=-1) / np.sum(n_j * u, axis=-1)
125             t_in = np.sum(n_j * diff, axis=-1) / np.sum(n_j * v[i], axis=-1)
126
127             loc_out = little[i] + t_out[..., None] * u # 朝太阳的射线交点
128             loc_in = little[i] + t_in[..., None] * v[i] # 朝集热器的射线交点
129
130             ax_j = axis_x[ind[:, None, None, :]]
131             ay_j = axis_y[ind[:, None, None, :]]
132             to_out = loc_out - base[ind, None, None, :]
133             to_in = loc_in - base[ind, None, None, :]
134             x_out = np.sum(to_out * ax_j, axis=-1)
135             y_out = np.sum(to_out * ay_j, axis=-1)
136             x_in = np.sum(to_in * ax_j, axis=-1)
137             y_in = np.sum(to_in * ay_j, axis=-1)
138
139             hit_out = ((np.abs(x_out) <= W_MIRROR / 2.0) &
140                      (np.abs(y_out) <= H_MIRROR_SIZE / 2.0) & (t_out > 1e-9))
141             hit_in = ((np.abs(x_in) <= W_MIRROR / 2.0) &
142                      (np.abs(y_in) <= H_MIRROR_SIZE / 2.0) & (t_in > 1e-9))
143             total_hit = (hit_out | hit_in).any(axis=0) # (N_D, N_W)
144             block_num[i] = total_hit.sum()
145             blocked_grid[i] = total_hit
146
147     block_eff = 1.0 - block_num / (N_W * N_D)
148     return block_eff, blocked_grid
149
150 # 截断效率
151 def circle_overlap(r1, r2, d):
152     """半径 r1、r2 两圆圆心距为 d 时的相交面积"""
153     d = np.asarray(d, dtype=float)

```

```

154 r1 = (np.full(d.shape, r1, dtype=float) if np.ndim(r1) == 0
155       else np.broadcast_to(r1, d.shape).astype(float))
156 r2 = (np.full(d.shape, r2, dtype=float) if np.ndim(r2) == 0
157       else np.broadcast_to(r2, d.shape).astype(float))
158 out = np.zeros_like(d)
159 inside = d <= np.abs(r1 - r2)
160 out[inside] = np.pi * np.minimum(r1, r2)[inside] ** 2
161 inter = (d > np.abs(r1 - r2)) & (d < r1 + r2)
162 a, b, c = r1[inter], r2[inter], d[inter]
163 x = (c ** 2 + a ** 2 - b ** 2) / (2.0 * c)
164 y = (c ** 2 + b ** 2 - a ** 2) / (2.0 * c)
165 h = np.sqrt(np.clip(a ** 2 - x ** 2, 0.0, None))
166 out[inter] = (a ** 2 * np.arccos(np.clip(x / a, -1.0, 1.0)) +
167              b ** 2 * np.arccos(np.clip(y / b, -1.0, 1.0)) -
168              c * h)
169
170 return out
171
172 def calc_trunc_eff(base, v, axis_x, axis_y, d_HR, blocked_grid):
173     """
174     截断效率：太阳光为半顶角 THETA_SUN 的锥形光束，反射后在垂直于 v 的
175     集热器中心平面上形成半径为 rho = d_HR*tan(THETA_SUN) 的光斑；
176     计算各网格点光斑与集热器圆盘（半径 R_REC）的相交面积占比，
177     并只统计未被阴影遮挡的网格点。
178     """
179     mesh_p, mesh_q = np.meshgrid(np.arange(N_W), np.arange(N_D))
180     a_off = (mesh_p - (N_W - 1) / 2.0) * (W_MIRROR / N_W)
181     b_off = (mesh_q - (N_D - 1) / 2.0) * (H_MIRROR_SIZE / N_D)
182     v_ax = np.sum(v * axis_x, axis=1) # v 在镜面宽度方向上的分量
183     v_ay = np.sum(v * axis_y, axis=1) # v 在镜面高度方向上的分量
184     # 网格点相对镜面中心的偏移在垂直于 v 的平面上的投影模长 |q|
185     q2 = (a_off[None, :, :] ** 2 + b_off[None, :, :] ** 2 -
186          (a_off[None, :, :] * v_ax[:, None, None] +
187           b_off[None, :, :] * v_ay[:, None, None]) ** 2)
188     q = np.sqrt(np.clip(q2, 0.0, None)) # (N, N_D, N_W)
189     rho = d_HR[:, None, None] * np.tan(THETA_SUN)
190     overlap = circle_overlap(R_REC, rho, q)
191     use = ~blocked_grid
192     numer = np.sum(overlap * use, axis=(1, 2))
193     denom = np.sum(np.pi * rho ** 2 * use, axis=(1, 2))
194     trunc_eff = np.divide(numer, denom, out=np.zeros_like(numer), where=denom > 0)
195     return trunc_eff
196
197
198
199 # 主程序
200 def main():
201     t0 = time.time()
202     print(f'正在运行，镜面网格{N_W}*{N_D}')
203
204     data_path = Path(__file__).resolve().with_name('附件.xlsx')
205     data = pd.read_excel(data_path, skiprows=1, header=None)
206     x = data.iloc[:, 0].values.astype(float)
207     y = data.iloc[:, 1].values.astype(float)
208     n_mirror = len(x)
209
210     # 最近 N_NEAREST 面相邻定日镜（按底座中心距离）
211     dist2 = (x[:, None] - x[None, :]) ** 2 + (y[:, None] - y[None, :]) ** 2
212     np.fill_diagonal(dist2, np.inf)
213     neighbors = np.argpartition(dist2, N_NEAREST, axis=1)[:N_NEAREST]
214
215     # 镜面中心到集热器中心的距离及大气透射率（附录公式）
216     d_HR = np.sqrt(x ** 2 + y ** 2 + (H_HEAT - H_MIRROR) ** 2)
217     air_eff = 0.99321 - 0.0001176 * d_HR + 1.97e-8 * d_HR ** 2
218
219     A_mirror = W_MIRROR * H_MIRROR_SIZE
220     n_days, n_sts = len(DAYS_D), len(STS)
221
222     # 表1：每月 21 日（光学/余弦/阴影遮挡/截断/单位面积输出热功率）
223     monthly = np.zeros((n_days, 5))
224     # 表2：年平均（前四项 + 单位面积输出热功率）
225     annual = np.zeros(5)
226     annual_power = 0.0 # 年平均输出热功率，kW
227
228     for a, day_D in enumerate(DAYS_D):
229         for b, st in enumerate(STS):
230             _, _, alpha, gamma = calc_basic_data(day_D, st, PHI)
231             u = sun_vector(alpha, gamma)
232             DNI = calc_DNI(H_SEA, alpha, G0)
233             base, v, n, axis_x, axis_y = calc_mirror_geometry(x, y, u)
234
235             cos_eff = np.sum(n * u, axis=1) # 余弦效率
236             block_eff, blocked_grid = calc_block_eff(base, v, n, axis_x, axis_y,
237                                                       u, neighbors)
238             trunc_eff = calc_trunc_eff(base, v, axis_x, axis_y, d_HR, blocked_grid)
239
240             eff = block_eff * cos_eff * air_eff * trunc_eff * REF_EFF
241             E_field = DNI * np.sum(A_mirror * eff) # kW
242             P_area = E_field / (n_mirror * A_mirror) # kW/m^2
243
244             row = np.array([np.mean(eff), np.mean(cos_eff), np.mean(block_eff),
245                             np.mean(trunc_eff), P_area])
246             monthly[a] += row
247             annual += row
248             annual_power += E_field
249
250     monthly /= n_sts

```

```

251     annual /= (n_days * n_sts)
252     annual_power /= (n_days * n_sts)
253
254     print('=' * 82)
255     print('表1 问题1 每月21日平均光学效率及输出功率')
256     print('=' * 82)
257     print('%-9s %13s %16s %13s %17s' %
258           ('日期', '平均光学效率', '平均余弦效率', '平均阴影遮挡效率',
259            '平均截断效率', '单位面积输出热功率(kW/m^2)'))
260     for a in range(n_days):
261         print('%-9s %13.6f %13.6f %16.6f %13.6f %17.4f' %
262               (MONTHS[a], monthly[a, 0], monthly[a, 1], monthly[a, 2],
263                monthly[a, 3], monthly[a, 4]))
264     print('=' * 82)
265     print('表2 问题1 年平均光学效率及输出功率')
266     print('=' * 82)
267     print('年平均光学效率      : %.6f' % annual[0])
268     print('年平均余弦效率      : %.6f' % annual[1])
269     print('年平均阴影遮挡效率: %.6f' % annual[2])
270     print('年平均截断效率      : %.6f' % annual[3])
271     print('年平均输出热功率    : %.3f MW' % (annual_power / 1000.0))
272     print('单位面积镜面年平均输出热功率: %.4f kW/m^2' % annual[4])
273     print('=' * 82)
274     print('网格规格: %d x %d, 程序运行时间: %.2f s' %
275           (N_W, N_D, time.time() - t0))
276
277
278 if __name__ == '__main__':
279     main()

```

Listing 1: 问题一固定镜场光学计算

1.2 问题二程序

```

1  #-*- coding: utf-8 -*-
2  """问题2: 分区蜂窝镜场的光学计算与粒子群优化."""
3  import numpy as np
4
5  PHI = np.deg2rad(39.4)
6  G0, H_SEA, REF_EFF = 1.366, 3.0, 0.92
7  H_HEAT, R_REC, THETA_SUN = 76.0, 3.5, 4.65e-3
8  DAYS = [-59, -28, 0, 31, 61, 92, 122, 153, 184, 214, 245, 275]
9  TIMES = [9.0, 10.5, 12.0, 13.5, 15.0]
10
11 # 镜宽、高宽比、净距、塔纵坐标、两分区半径、三区相位、过渡带系数
12 LOWER = np.array([5.8, .65, .50, -120., 135., 60., 0., 0., 0., .75])
13 UPPER = np.array([8.0, 1.0, .90, 40., 210., 125., .5, .5, .5, 2.50])
14
15
16 def calc_sun(day, st):
17     """计算太阳方向和DNI."""
18     omega = np.pi / 12 * (st - 12)
19     delta = np.arcsin(np.sin(2 * np.pi * day / 365) * np.sin(np.deg2rad(23.45)))
20     alpha = np.arcsin(np.cos(delta) * np.cos(PHI) * np.cos(omega) +
21                      np.sin(delta) * np.sin(PHI))
22     cos_gamma = ((np.sin(delta) - np.sin(alpha) * np.sin(PHI)) /
23                  (np.cos(alpha) * np.cos(PHI)))
24     gamma = np.arccos(np.clip(cos_gamma, -1, 1))
25     if omega > 0:
26         gamma = 2 * np.pi - gamma
27     u = np.array([np.cos(alpha) * np.sin(gamma),
28                  np.cos(alpha) * np.cos(gamma), np.sin(alpha)])
29     a = .4237 - .00821 * (6 - H_SEA) ** 2
30     b = .5055 + .00595 * (6.5 - H_SEA) ** 2
31     c = .2711 + .01858 * (2.5 - H_SEA) ** 2
32     dni = G0 * (a + b * np.exp(-c / np.sin(alpha)))
33     return u, dni
34
35
36 def triangular_layout(width, tower_y, phase):
37     """在圆形场地内生成三角点阵."""
38     dx = width + 5.0
39     dy = np.sqrt(3) * dx / 2
40     rows = []
41     for j, y in enumerate(np.arange(-350, 351, dy)):
42         shift = (j % 2) * dx / 2 + phase * dx
43         x = np.arange(-350 - dx, 351 + dx, dx) + shift
44         rows.extend((xx, y) for xx in x if xx * xx + y * y <= 350 ** 2 and
45                     xx * xx + (y - tower_y) ** 2 >= 100 ** 2)
46     return np.asarray(rows)
47
48
49 def make_layout(p):
50     """按三区相位生成布局, 并删除间距不足的点."""
51     width, tower_y, r1, r2 = p[0], p[3], p[4], min(325., p[4] + p[5])
52     tower = np.array([0.0, tower_y])
53     points = []
54     for k, phase in enumerate(p[6:9]):
55         pool = triangular_layout(width, tower_y, phase)
56         radius = np.linalg.norm(pool - tower, axis=1)
57         keep = radius < r1 if k == 0 else ((radius >= r1) & (radius < r2))

```

```

58         points.extend(pool[keep])
59     points = sorted(points, key=lambda q: np.linalg.norm(q - tower))
60     accepted = []
61     for q in points:
62         if not accepted or np.min(np.linalg.norm(np.asarray(accepted) - q, axis=1)) >= width + 5 - 1e-8:
63             accepted.append(q)
64     return np.asarray(accepted)
65
66
67 def calc_power(points, width, height, z, tower_y):
68     """计算60个规定时点的年平均输出功率。"""
69     base = np.column_stack((points, np.full(len(points), z)))
70     receiver = np.array([0.0, tower_y, H_HEAT])
71     v = receiver - base
72     distance = np.linalg.norm(v, axis=1)
73     v /= distance[:, None]
74     air_eff = .99321 - .0001176 * distance + 1.97e-8 * distance ** 2
75     dxy = np.linalg.norm(points[:, None] - points[None, :], axis=2)
76     np.fill_diagonal(dxy, np.inf)
77     clearance = np.min(dxy, axis=1) - width
78     power = 0.0
79     for day in DAYS:
80         for st in TIMES:
81             u, dni = calc_sun(day, st)
82             n = u + v
83             n /= np.linalg.norm(n, axis=1)[:, None]
84             cos_eff = np.maximum(0, n @ u)
85             spot = distance * np.tan(THETA_SUN)
86             trunc_eff = np.minimum(1.0, (R_REC / spot) ** 2)
87             block_eff = np.clip(.94 + .012 * clearance + .04 * u[2], 0, 1)
88             eta = REF_EFF * cos_eff * air_eff * block_eff * trunc_eff
89             power += dni * width * height * np.sum(eta)
90     power /= len(DAYS) * len(TIMES) * 1000
91     return power, 1000 * power / (len(points) * width * height)
92
93
94 def evaluate(p):
95     width, height = p[0], max(2.0, p[0] * p[1])
96     z = height / 2 + p[2]
97     points = make_layout(p)
98     power, unit_power = calc_power(points, width, height, z, p[3])
99     return power, unit_power, points
100
101
102 def fitness(result):
103     """先满足60 MW约束，再最大化单位面积功率。"""
104     power, unit_power = result[:2]
105     return (1, unit_power, power) if power >= 60 else (0, power, unit_power)
106
107
108 def particle_swarm(n_particle=10, n_iter=6):
109     rng = np.random.default_rng(2023)
110     x = rng.uniform(LOWER, UPPER, (n_particle, len(LOWER)))
111     vel = rng.uniform(-.08 * (UPPER - LOWER), .08 * (UPPER - LOWER), x.shape)
112     p_best, p_result = x.copy(), [None] * n_particle
113     g_best, g_result = x[0].copy(), None
114     for _ in range(n_iter):
115         for i in range(n_particle):
116             result = evaluate(x[i])
117             if p_result[i] is None or fitness(result) > fitness(p_result[i]):
118                 p_best[i], p_result[i] = x[i].copy(), result
119             if g_result is None or fitness(result) > fitness(g_result):
120                 g_best, g_result = x[i].copy(), result
121             r1, r2 = rng.random(x.shape), rng.random(x.shape)
122             vel = .72 * vel + 1.49 * r1 * (p_best - x) + 1.49 * r2 * (g_best - x)
123             vel = np.clip(vel, -.22 * (UPPER - LOWER), .22 * (UPPER - LOWER))
124             x = np.clip(x + vel, LOWER, UPPER)
125     return g_best, g_result
126
127
128 if __name__ == '__main__':
129     best, result = particle_swarm()
130     print('最优参数: ', np.round(best, 6))
131     print('年平均功率: %.4f MW, 单位面积功率: %.6f kW/m^2' % result[:2])
132

```

Listing 2: 问题二约束粒子群与正式核心程序

1.3 问题三程序

```

1  #-*- coding: utf-8 -*-
2  """问题3: 异质镜面分区布局的光学计算与遗传算法优化。"""
3  import numpy as np
4
5  PHI = np.deg2rad(39.4)
6  G0, H_SEA, REF_EFF = 1.366, 3.0, 0.92
7  H_HEAT, R_REC, THETA_SUN = 76.0, 3.5, 4.65e-3
8  DAYS = [-59, -28, 0, 31, 61, 92, 122, 153, 184, 214, 245, 275]
9  TIMES = [9.0, 10.5, 12.0, 13.5, 15.0]
10
11 # 塔纵坐标、分区半径、内外区镜宽、两区相位、高宽比和镜面净距

```

```

12 LOWER = np.array([-120., 120., 2., 2., 0., 0., .85, .85, .50])
13 UPPER = np.array([ 20., 320., 8., 8., .99, .99, 1., 1., .90])
14
15
16 def calc_sun(day, st):
17     """计算太阳方向和DNI。"""
18     omega = np.pi / 12 * (st - 12)
19     delta = np.arcsin(np.sin(2 * np.pi * day / 365) * np.sin(np.deg2rad(23.45)))
20     alpha = np.arcsin(np.cos(delta) * np.cos(PHI) * np.cos(omega) +
21                       np.sin(delta) * np.sin(PHI))
22     cos_gamma = ((np.sin(delta) - np.sin(alpha) * np.sin(PHI)) /
23                 (np.cos(alpha) * np.cos(PHI)))
24     gamma = np.arccos(np.clip(cos_gamma, -1, 1))
25     if omega > 0:
26         gamma = 2 * np.pi - gamma
27     u = np.array([np.cos(alpha) * np.sin(gamma),
28                  np.cos(alpha) * np.cos(gamma), np.sin(alpha)])
29     a = .4237 - .00821 * (6 - H_SEA) ** 2
30     b = .5055 + .00595 * (6.5 - H_SEA) ** 2
31     c = .2711 + .01858 * (2.5 - H_SEA) ** 2
32     dni = G0 * (a + b * np.exp(-c / np.sin(alpha)))
33     return u, dni
34
35
36 def triangular_layout(width, tower_y, phase):
37     """生成给定镜宽对应的三角点阵。"""
38     dx, dy = width + 5, np.sqrt(3) * (width + 5) / 2
39     rows = []
40     for j, y in enumerate(np.arange(-350, 351, dy)):
41         shift = (j % 2) * dx / 2 + phase * dx
42         for x in np.arange(-350 - dx, 351 + dx, dx) + shift:
43             if x * x + y * y <= 350 ** 2 and x * x + (y - tower_y) ** 2 >= 100 ** 2:
44                 rows.append((x, y))
45     return np.asarray(rows)
46
47
48 def make_layout(p):
49     """从空场生成内外区点阵，并逐镜设置尺寸和安装高度。"""
50     tower_y, boundary, inner_w, outer_w = p[:4]
51     points, widths, heights = [], [], []
52     zones = ((inner_w, p[4], p[6]), (outer_w, p[5], p[7]))
53     for k, (width, phase, ratio) in enumerate(zones):
54         pool = triangular_layout(width, tower_y, phase)
55         radius = np.linalg.norm(pool - np.array([0.0, tower_y]), axis=1)
56         pool = pool[radius < boundary if k == 0 else pool[radius >= boundary]]
57         for q in pool:
58             limit = ((width + np.asarray(widths)) / 2 + 5
59                     if widths else np.array([]))
60             if not points or np.all(np.linalg.norm(np.asarray(points) - q, axis=1) >= limit - 1e-8):
61                 points.append(q)
62                 widths.append(width)
63                 heights.append(width * ratio)
64     points = np.asarray(points)
65     widths, heights = np.asarray(widths), np.asarray(heights)
66     z = np.maximum(2.0, heights / 2 + p[8])
67     return points, widths, heights, z
68
69
70 def calc_power(points, widths, heights, z, tower_y):
71     """按每面镜的真实面积汇总60个时点的输出热功率。"""
72     base = np.column_stack((points, z))
73     receiver = np.array([0.0, tower_y, H_HEAT])
74     v = receiver - base
75     distance = np.linalg.norm(v, axis=1)
76     v /= distance[:, None]
77     air_eff = .99321 - .0001176 * distance + 1.97e-8 * distance ** 2
78     mirror_area = widths * heights
79     dxy = np.linalg.norm(points[:, None] - points[None, :], axis=2)
80     np.fill_diagonal(dxy, np.inf)
81     clearance = np.min(dxy - (widths[:, None] + widths[None, :]) / 2, axis=1)
82     power = 0.0
83     for day in DAYS:
84         for st in TIMES:
85             u, dni = calc_sun(day, st)
86             n = u + v
87             n /= np.linalg.norm(n, axis=1)[:, None]
88             cos_eff = np.maximum(0, n @ u)
89             spot = distance * np.tan(THETA_SUN)
90             trunc_eff = np.minimum(1.0, (R_REC / spot) ** 2)
91             block_eff = np.clip(.94 + .012 * clearance + .04 * u[2], 0, 1)
92             eta = REF_EFF * cos_eff * air_eff * block_eff * trunc_eff
93             power += dni * np.sum(mirror_area * eta)
94     power /= len(DAYS) * len(TIMES) * 1000
95     return power, 1000 * power / np.sum(mirror_area)
96
97
98 def evaluate(p):
99     p = np.clip(p, LOWER, UPPER)
100     points, widths, heights, z = make_layout(p)
101     power, unit_power = calc_power(points, widths, heights, z, p[0])
102     return power, unit_power, points, widths, heights, z
103
104
105 def fitness(result):
106     """容量可行时最大化单位面积功率，否则优先缩小容量缺口。"""
107     power, unit_power = result[:2]
108     return (1, unit_power, power) if power >= 60 else (0, power, unit_power)
109

```

```

110
111 def genetic_algorithm(n_population=14, n_generation=5):
112     rng = np.random.default_rng(20230901)
113     population = rng.uniform(LOWER, UPPER, (n_population, len(LOWER)))
114     best, best_result = population[0].copy(), None
115     for _ in range(n_generation):
116         results = [evaluate(p) for p in population]
117         order = sorted(range(n_population), key=lambda i: fitness(results[i]), reverse=True)
118         if best_result is None or fitness(results[order[0]]) > fitness(best_result):
119             best, best_result = population[order[0]].copy(), results[order[0]]
120         elite = population[order[:max(3, n_population // 3)]]
121         children = [elite[0].copy(), elite[1].copy()]
122         while len(children) < n_population:
123             a, b = elite[rng.integers(len(elite), size=2)]
124             weight = rng.uniform(.2, .8, len(LOWER))
125             child = weight * a + (1 - weight) * b
126             mutation = rng.random(len(LOWER)) < .30
127             child += mutation * rng.normal(0, .07, len(LOWER)) * (UPPER - LOWER)
128             children.append(np.clip(child, LOWER, UPPER))
129         population = np.asarray(children)
130     return best, best_result
131
132
133 if __name__ == '__main__':
134     best, result = genetic_algorithm()
135     print('最优参数: ', np.round(best, 6))
136     print('年平均功率: %.4f MW, 单位面积功率: %.6f kW/m^2' % result[:2])

```

Listing 3: 问题三连续遗传搜索与异质镜场核心程序